translation_attention_pytorch

December 3, 2025

1 Seq2Seq with Attention for Machine Translation (and a Mystery Language)

This notebook is the **attention-enhanced** version of our seq2seq machine translation tutorial. It assumes you've already seen a basic encoder–decoder LSTM *without* attention in a separate "vanilla seq2seq" notebook, and focuses on how adding attention changes the architecture, training dynamics, and translation quality.

We work with a small multi-parallel corpus containing sentences in:

- Danish
- English
- German
- Finnish
- Spanish
- A "mystery" coded language

This data was adapted from the European Parliament Proceedings Parallel Corpus 1996-2011.

The notebook has two main goals:

1. **Build and inspect a seq2seq + attention translation model (English → Spanish)**
   - Load and preprocess parallel English–Spanish data
   - Build vocabularies and PyTorch `Dataset` / `DataLoader` objects
   - Implement an encoder–decoder LSTM **with dot-product attention**
   - Train the model with teacher forcing, dropout, and early stopping
   - Visualize training vs validation loss
   - Evaluate with `sacrebleu` and inspect example translations (including variants with and without `<UNK>` filtering)

- Compare the behavior of this attention model to the simpler non-attention baseline

2. **Use attention-based translation models to identify the mystery language**
   - For each candidate source language (Danish, English, German, Finnish, Spanish), train a seq2seq+attention model to translate from that language into the mystery language
   - Compute BLEU scores on held-out test data for each (candidate → mystery) pair
   - Compare scores to determine which candidate language is most compatible with the mystery stream
   - Interpret the results in terms of a codebook-style cipher built on top of an unknown language.

By the end of the notebook, you will have:

- A working **seq2seq + attention** translation model for English → Spanish
- Plots of training and validation loss for multiple attention-based language pairs
- BLEU-based evidence of whatt the "mystery" language is a systematically encoded version of
- A concrete comparison point to the simpler, non-attention seq2seq model from the baseline notebook

1.1 TO RUN THIS NOTEBOOK YOU MUST MAKE THE DATA FILES AVAILABLE

For this notebook we use pre-tokenized parallel sentences stored in three pickled dictionaries:

- `mt_train_sentences.pkl`
- `mt_val_sentences.pkl`
- `mt_test_sentences.pkl`

Each file is a Python `dict` mapping language names to lists of tokenized sentences, for example:

- `"english"` → list of English sentences (each as a list of tokens)
- `"spanish"` → list of Spanish sentences
- `"danish"`, `"german"`, `"finnish"`, `"mystery"`, etc.

  Folder structure:
  Put all three `.pkl` files into a folder called `data` in the same directory as this notebook:

```
your_project/
   translation_attention.ipynb
   data/
       mt_train_sentences.pkl
       mt_val_sentences.pkl
```

```
mt_test_sentences.pkl
```

The loading code expects paths like:

```
with open("data/mt_train_sentences.pkl", "rb") as f:
    train_dict = pickle.load(f)
```

Once these files are in place, we can:

- Load the three dictionaries for train/validation/test.
- Build vocabularies over the chosen source and target languages.
- Wrap the tokenized sentence pairs in a `TranslationDataset` and use PyTorch `DataLoader`s (with a custom collate function) to produce padded tensors that will be fed into our seq2seq + attention model.

2 1. Foundations

2.1 What is attention?

In this notebook we use **encoder–decoder attention**, the classic mechanism introduced for sequence-to-sequence models, not Transformer-style self-attention.

At a high level, attention lets the decoder decide which parts of the encoder's hidden states are most relevant when predicting each output token.

- Attention is typically used in encoder–decoder architectures such as neural machine translation.
- On each decoding step, we compute a set of *attention scores* over the encoder's hidden states.
- These scores are normalized to give *attention weights*, which tell us how much emphasis to place on each encoder hidden state.
- The weighted sum of encoder states is called the *context vector*; it is fed into the decoder (often concatenated with the decoder hidden state) to make the next prediction.
- Attention scores can be computed in several ways, including dot-product, bilinear forms, or small feed-forward networks (often called "additive" attention).

In the sections that follow, we will implement dot-product attention and see how it improves a simple seq2seq translation model.

2.2 Does translation direction matter (A→B vs B→A)?

Even when we use the same architecture and the same number of parallel sentences, we should *not* expect a seq2seq-with-attention model to perform identically in both directions (A→B vs B→A). Natural languages are not in a neat one-to-one correspondence: vocabulary sizes differ, phrases of length N in one language may map to phrases of length M in the other, and some semantic distinctions that are lexicalized in one language are collapsed or paraphrased in the other.

This often makes one direction more "lossy" than the other. For example, in Spanish the word `prima` means "female cousin." In real usage, translating to English, this would usually just become `cousin`, because explicitly saying "female cousin" ("I went for a walk with my female cousin") sounds marked or awkward. A seq2seq model trained on actual usage would likely learn the more natural `prima → cousin` mapping. But now, going back from English to Spanish, the model has

to choose between `prima` (female) and `primo` (male), even though that gender information was thrown away when we went into English.

In the other direction, consider the English phrasal verb `look forward to`, which has no single-word equivalent in Spanish. A very common translation is `esperar`, which can mean "to wait" or "to hope," so "I'm looking forward to seeing you" often becomes something like "espero verte." Information is lost: "looking forward" implies positive anticipation and high likelihood, not just desire. Testing this in Google Translate is interesting, as the translations reach a sort of equilibrium where the phrase is mutually translatable after the information loss has occurred. In this example in Google Translate, `I look forward to seeing you` → `Espero verte pronto` → `I hope to see you soon`.

These examples illustrate the general point: because languages differ in how granular or coarse their lexical choices are, and because some semantic distinctions cannot be perfectly preserved, the A→B mapping and the B→A mapping do not have the same difficulty. As a result, we should not expect a seq2seq+attention model to achieve the same performance range in both directions, even on the same parallel corpus.

2.3 What happens if we scramble all the characters?

Consider a word-level seq2seq model with attention trained on a parallel corpus. What happens if we systematically scramble the characters in the source language before tokenization?

2.3.1 Case 1: Scramble characters, but never introduce whitespace

In the first scenario, we replace every distinct non-whitespace character in the source language with a different distinct non-whitespace character. For example, `h → s`, `e → w`, `l → o`, `o → x`, so that `"hello"` becomes `"swoox"`. The mapping is one-to-one and never produces spaces.

From the model's perspective, nothing fundamental has changed. It still sees a sequence of *word tokens* separated by whitespace, and the vocabulary is the same size—each original word just has a different spelling. During training, the model can learn embeddings for `"swoox"` that play the same role `"hello"` used to play. Since translation operates on a per-word basis and we haven't created new word boundaries, we should expect performance to be essentially unchanged.

Another way to see this is to notice that this is just a simple substitution cipher at the character level (like a Caesar cipher). Frequency analysis can recover the original text because the distribution of words and their co-occurrences is untouched. All we've done is rename the characters; the underlying sentences and their meanings are the same, so a word-level seq2seq model can, in principle, learn exactly the same mapping.

2.3.2 Case 2: Scramble characters and allow whitespace

In the second scenario, we again replace every distinct character with a different distinct character, but now the replacement may include whitespace. That is, some letters might be turned into spaces. The mapping is still one-to-one at the character level, but now a single original word can be transformed into multiple whitespace-separated chunks.

If the language uses spaces as word delimiters (like English, Spanish, etc.), this is a big problem: tokenization changes. A single word in the original corpus can become several "words" in the

scrambled corpus, and the one-to-one correspondence between source words and their scrambled forms is broken. For example, imagine the letter `p` is mapped to a space:

- Original: `I own a campsite`
- After mapping `p → space` (before we scramble the other characters):
  `I own a cam site`
  Now `campsite` has been split into `cam` and `site`, which clearly has a different meaning.
- After scrambling the remaining letters into other non-whitespace symbols, we still see that the tokenization pattern differs (e.g., `O gcf a ead lomt` vs something like `O gcf a eadhlomt` if no space had been introduced).

For a space-delimited language, this means:

- The model now has to learn mappings from longer, more fragmented source sequences to target sentences.
- The neat word-level correspondences in the original parallel data are replaced by more complex many-token-to-one-word mappings.
- Semantic cues that were neatly packaged in single word tokens are now scattered across multiple tokens.

Once whitespace can be generated by the scrambling, "single words become multiple words," and the clean one-to-one mapping at the word level is lost. This makes the translation task strictly harder, so we should expect performance to decrease in this second scenario.

3 2. Machine Translation: Sequence-to-sequence translation with attention

In this part of the notebook we will build and experiment with a neural machine translation system implemented in PyTorch. The core building block is an RNN-based encoder–decoder model with LSTM units and a dot-product attention mechanism.

The encoder reads a source sentence one token at a time and produces a sequence of hidden states. The decoder then generates the target sentence, using attention to decide how much to focus on each encoder hidden state at every decoding step. Intuitively, attention lets the model "look back" at different parts of the source sentence while producing each target word.

We will first train this model on an English → Spanish parallel corpus, monitor train/validation loss, and inspect example translations to see what the model has learned. Then we will reuse exactly the same architecture to translate from several different candidate source languages into a `mystery` language, compare their BLEU scores, and use those results to reason about what the mystery language must be. Along the way, you will see how the encoder, decoder, and attention components work together in a practical, end-to-end translation system.

[ ]:
```python
# Core Python and typing utilities
import copy
import pickle
import random
import subprocess
import sys
from typing import Dict, List, Tuple
```

```
# Data science and plotting libraries
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

# PyTorch components
import torch
from torch import as_tensor, nn, optim
from torch.nn import BCEWithLogitsLoss, CrossEntropyLoss
from torch.nn.utils.rnn import (
    pack_padded_sequence,
    pad_packed_sequence,
    pad_sequence,
)
from torch.utils.data import DataLoader, Dataset
from torch.optim import Adam

# BLEU evaluation (install sacrebleu on the fly if needed)
try:
    import sacrebleu
except ImportError:
    subprocess.check_call([sys.executable, "-m", "pip", "install", "-q",␣
 ↪"sacrebleu"])
    import sacrebleu
```

3.1 Machine translation Setup

We will now put the encoder–decoder with attention to work on a genuine sequence-to-sequence task: machine translation.

Our plan is:

- First, train the model on an English → Spanish parallel corpus to understand its behavior on a single language pair (loss curves, example translations, BLEU scores).
- Then, reuse the same architecture to translate from several different candidate source languages into a `mystery` language, and compare their BLEU scores. The relative performance across language pairs will help us reason about the identity of the mystery language.

3.1.1 Parallel translation data

For these experiments, we use pre-tokenized, pickled datasets.
Each split is stored in a separate file (documentation on what Pickling is):

- `mt_train_sentences.pkl`
- `mt_val_sentences.pkl`
- `mt_test_sentences.pkl`

Each file contains a Python dictionary mapping language names to lists of tokenized sentences. The available languages are:

- Danish
- English
- German
- Finnish
- Spanish
- `mystery` (an obfuscated or lossy encoded language)

The general structure of each file looks like this:

```
{
    "danish": [
        ["token", "token", "token", ...],
        ["token", "token", "token", ...],
        ...
    ],
    "english": [
        ["token", "token", "token", ...],
        ["token", "token", "token", ...],
        ...
    ],
    "mystery": [
        ["token", "token", "token", ...],
        ["token", "token", "token", ...],
        ...
    ],
    # etc.
}
```

The key indicates the language, and the n-th sentence in each list forms a parallel set of translations across languages. For example, entry `i` in `"english"` and entry `i` in `"spanish"` are translations of the same underlying sentence; likewise for `"english"` and `"mystery"`, `"danish"` and `"mystery"`, and so on.

In the code below, we will:

- Load these pickled dictionaries for the train, validation, and test splits.
- Build vocabularies from the tokenized sentences.
- Create PyTorch `Dataset` and `DataLoader` objects that yield padded mini-batches suitable for our seq2seq + attention model.

As you go, it's worth inspecting a few batches from the dataloaders (for example, `next(iter(train_loader))`) to get an intuition for what the model actually sees during training.

3.1.2 Building vocabularies

The helpers below are used to turn raw tokenized sentences into vocabularies that our models can work with.

- `get_word_counts` computes how often each token appears in a corpus.
  We use these counts to tell us which words are common enough to keep as separate entries,

and which are so rare that it's better to group them into a single `<UNK>` (unknown) token. This keeps the vocabulary manageable and reduces overfitting to one-off typos or extremely rare words.

- `generate_vocab` then assigns each token an integer ID, reserving special indices for `<PAD>`, `<EOS>`, and `<UNK>`, and mapping very rare tokens to `<UNK>` based on `min_freq`.

Later, we'll use these vocabularies to convert sentences into sequences of indices for our embedding layers and dataloaders.

[ ]:
```python
# Special token indices used throughout the notebook
PADDING_IDX = 0   # padding for shorter sequences in a batch
EOS_IDX = 1       # end-of-sequence marker
UNK_IDX = 2       # out-of-vocabulary / rare words

# Helper functions for building vocabularies from tokenized text

def get_word_counts(sentences: List[List[str]]) -> Dict[str, int]:
    """
    Count token frequencies in a tokenized corpus.

    Args:
        sentences: List of sentences, where each sentence is a list of tokens.

    Returns:
        A dictionary mapping each token to its frequency in the corpus.
    """
    word_counts: Dict[str, int] = {}
    for sent in sentences:
        for word in sent:
            if word in word_counts:
                word_counts[word] += 1
            else:
                word_counts[word] = 1
    return word_counts


def generate_vocab(word_counts: Dict[str, int], min_freq: int) ->␣
↪Tuple[Dict[str, int], int]:
    """
    Build a token → index vocabulary from token frequency counts.

    Args:
        word_counts: A dictionary mapping tokens to their counts.
        min_freq:    Minimum frequency required for a token to be kept
                     as its own entry in the vocabulary. Tokens that occur
                     <= min_freq times are mapped to <UNK>.

    Returns:
```

```
        vocab:    A dictionary mapping tokens to integer indices.
        vocab_sz: The size of the vocabulary (number of distinct indices).
    """
    # Sort tokens by descending frequency
    sorted_words = sorted(word_counts.items(), key=lambda x: x[1], reverse=True)

    # Reserve indices 0, 1, 2 for special tokens (<PAD>, <EOS>, <UNK>)
    vocab = {
        word: i + 3
        for i, (word, count) in enumerate(sorted_words)
        if count > min_freq
    }

    # Add special tokens
    vocab["<PAD>"] = PADDING_IDX
    vocab["<EOS>"] = EOS_IDX
    vocab["<UNK>"] = UNK_IDX

    return vocab, len(vocab)
```

3.1.3 Loading parallel data and building a translation dataset

We now load the parallel translation data for the train, validation, and test splits from the pickled dictionaries described above. For our first experiment, we will focus on the English → Spanish pair.

The `TranslationDataset` class wraps these tokenized sentence pairs into a PyTorch `Dataset`:

- On the training split, it builds separate source and target vocabularies from word counts (using `min_freq` to decide which words become `<UNK>`).
- On the validation and test splits, it reuses the same vocabularies as the training set so that indices are consistent across splits.
- The `_get_idx_dataset` helper converts token sequences to index sequences and appends/prepends special `<EOS>` markers:
  - Source sentences get an `<EOS>` token at the end.
  - Target sentences get `<EOS>` both at the beginning (used as a start-of-sequence token) and at the end.

At the end, we create three dataset objects: `train_ds`, `val_ds`, and `test_ds` for English → Spanish translation.

[ ]:
```
# Load tokenized parallel sentences for each split
with open("data/mt_train_sentences.pkl", "rb") as f:
    train_dict = pickle.load(f)

with open("data/mt_val_sentences.pkl", "rb") as f:
    val_dict = pickle.load(f)

with open("data/mt_test_sentences.pkl", "rb") as f:
    test_dict = pickle.load(f)
```

```python
class TranslationDataset(Dataset):
    """
    A simple dataset for parallel sentence pairs (source → target).

    If no vocabularies are provided, this dataset will:
      - Build a source vocabulary from the source sentences.
      - Build a target vocabulary from the target sentences.

    If vocabularies are provided (e.g., for val/test), it will reuse them
    so that token indices are consistent across splits.
    """

    def __init__(
        self,
        source: List[List[str]],
        target: List[List[str]],
        min_freq: int = 1,
        source_vocab: Dict[str, int] | None = None,
        target_vocab: Dict[str, int] | None = None,
    ) -> None:
        super().__init__()

        # Build vocabularies from scratch (typically for the training set)
        if source_vocab is None and target_vocab is None:
            self.source_vocab, self.source_vocab_size = generate_vocab(
                get_word_counts(source),
                min_freq,
            )
            self.target_vocab, self.target_vocab_size = generate_vocab(
                get_word_counts(target),
                min_freq,
            )
        else:
            # Reuse existing vocabularies (for validation / test)
            self.source_vocab = source_vocab
            self.source_vocab_size = len(source_vocab)

            self.target_vocab = target_vocab
            self.target_vocab_size = len(target_vocab)

        # Convert all token sequences to index sequences once up front
        self.source, self.target = self._get_idx_dataset(
            source,
            target,
            self.source_vocab,
```

```
            self.target_vocab,
        )

    def __len__(self) -> int:
        return len(self.target)

    def __getitem__(self, idx: int) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Return a single (source_indices, target_indices) pair as tensors.
        """
        return torch.tensor(self.source[idx]), torch.tensor(self.target[idx])

    def _get_idx_dataset(
        self,
        source: List[List[str]],
        target: List[List[str]],
        source_vocab: Dict[str, int],
        target_vocab: Dict[str, int],
    ) -> Tuple[List[List[int]], List[List[int]]]:
        """
        Map tokenized source/target sentences to index sequences, adding EOS␣
↪markers.

        - Source sentences: [w1, w2, ..., wN, <EOS>]
        - Target sentences: [<EOS>, w1, w2, ..., wM, <EOS>]
          (here we reuse <EOS> as a start-of-sequence token for the decoder)
        """
        source_toks: List[List[int]] = []
        target_toks: List[List[int]] = []

        for src, tar in zip(source, target):
            # Map source tokens to indices, then append EOS
            src_ids = [
                source_vocab[token]
                if token in source_vocab
                else source_vocab["<UNK>"]
                for token in src
            ]
            src_ids.append(source_vocab["<EOS>"])

            # For the target, prepend EOS as a start token, then map tokens,␣
↪then append EOS
            tar_ids = [target_vocab["<EOS>"]]
            tar_ids.extend(
                target_vocab[token]
                if token in target_vocab
                else target_vocab["<UNK>"]
```

```
                for token in tar
            )
            tar_ids.append(target_vocab["<EOS>"])

            source_toks.append(src_ids)
            target_toks.append(tar_ids)

        return source_toks, target_toks


# English → Spanish datasets for our first experiment
train_ds = TranslationDataset(
    train_dict["english"],
    train_dict["spanish"],
)

test_ds = TranslationDataset(
    test_dict["english"],
    test_dict["spanish"],
    source_vocab=train_ds.source_vocab,
    target_vocab=train_ds.target_vocab,
)

val_ds = TranslationDataset(
    val_dict["english"],
    val_dict["spanish"],
    source_vocab=train_ds.source_vocab,
    target_vocab=train_ds.target_vocab,
)
```

3.1.4 Inspecting parallel examples across languages

Before training the model, it helps to sanity-check the parallel corpus and see how sentences align across languages so that we can make sure everything is lined up, and see if we can even recognize our "mystery language" (spoiler alert–we can't).

The helper function below, `show_parallel_examples`, prints a few parallel sentence sets from one of the splits (`train_dict`, `val_dict`, or `test_dict`). For each selected index, it shows the corresponding sentence in:

- Danish
- English
- German
- Finnish

- Spanish

- `mystery`

You can either sample random indices (using an optional random seed for reproducibility) or view the first few examples in order. This is helpful for:

- Confirming that all language lists have the same length and are truly aligned.
- Getting an intuition for what the `mystery` language looks like relative to the others (gobbledygook!)

[ ]:
```python
# Utility to inspect aligned sentences across all languages in a split
def show_parallel_examples(
    data_dict: Dict[str, List[List[str]]],
    n_examples: int = 5,
    random_sample: bool = True,
    seed: int | None = None,
) -> None:
    """
    Print a few parallel sentence sets across all explicit languages plus the
    mystery language, using the provided data_dict (e.g., train_dict, val_dict,
    or test_dict).

    Args:
        data_dict:      Dictionary mapping language names to lists of tokenized
                        sentences (as loaded from the .pkl files).
        n_examples:     Number of parallel examples to display.
        random_sample:  If True, sample random indices; if False, use the first
                        n_examples in order.
        seed:           Optional random seed for reproducible sampling.
    """
    langs = ["danish", "english", "german", "finnish", "spanish", "mystery"]
    pretty = {
        "danish":  "Danish",
        "english": "English",
        "german":  "German",
        "finnish": "Finnish",
        "spanish": "Spanish",
        "mystery": "Mystery",
    }

    num_sentences = len(data_dict["english"])
    n_examples = min(n_examples, num_sentences)

    if seed is not None:
        random.seed(seed)

    # Choose which sentence indices to display
```

```
    if random_sample:
        indices = random.sample(range(num_sentences), n_examples)
    else:
        indices = list(range(n_examples))

    print("=== Parallel examples across languages ===")
    for ex_idx, sent_idx in enumerate(indices, start=1):
        print(f"\n--- Example {ex_idx} (index {sent_idx}) ---")
        for lang in langs:
            tokens = data_dict[lang][sent_idx]
            text = " ".join(tokens)
            print(f"{pretty[lang]}: {text}")
        print("-" * 60)


# Example: inspect a few random parallel sentences from the test split
show_parallel_examples(
    test_dict,
    n_examples=5,
    random_sample=True,
    seed=42,
)
```

```
=== Parallel examples across languages ===

--- Example 1 (index 654) ---
Danish: jeg er sikker på at den får positive virkninger for bulgarien og
rumænien
English: i am convinced that it will be positive for bulgaria and romania
German: ich bin sicher dass sie für bulgarien und rumänien positiv sein wird
Finnish: olen vakuuttunut siitä että laajentumisesta tulee myönteinen asia myös
bulgarialle ja romanialle
Spanish: estoy seguro de sus efectos positivos para bulgaria y rumanía
Mystery: rac neki izjk f hov khv qxmaj op xufg bws s ot
------------------------------------------------------------

--- Example 2 (index 114) ---
Danish: i er hverken interesseret i demokrati eller ægte demokratiske valg
English: you are not interested in democracy or genuine democratic elections
German: sie sind an demokratie bzw echten demokratischen wahlen nicht
interessiert
Finnish: te ette ole kiinnostuneita demokratiasta tai aidosti demokraattisista
vaaleista
Spanish: a ustedes no les interesan ni la democracia ni unas elecciones
verdaderamente democráticas
Mystery: wpzh gkxe qw w ma elaw ql qrzhw c vsul tduwf uefo a wah cnprf c vsul
tduwf xt zn ukio xuzgm bugla
```

```
------------------------------------------------------------

--- Example 3 (index 25) ---
Danish: vi står midt i en meget alvorlig krise med hensyn til legitimitet
English: we are in the middle of an extremely serious crisis of legitimacy
German: wir haben es mit einer legitimationskrise schwersten ausmaßes zu tun
Finnish: olemme äärimmäisen vakavan legitimiteettikriisin kourissa
Spanish: nos encontramos en mitad de una crisis de legitimidad muy grave
Mystery: qgky admsx jdly u jeak izjk f mpjx dv mayek t izjk f zcqs flrp jsl xevd
vags au
------------------------------------------------------------

--- Example 4 (index 281) ---
Danish: her har du ret doris pack
English: you are right there mrs pack
German: da hast du recht doris pack
Finnish: siinä olet oikeassa doris pack
Spanish: doris pack en eso tienes razón
Mystery: qrm rhk jdly bo z wcq d izc ofr zv dwb
------------------------------------------------------------

--- Example 5 (index 250) ---
Danish: de er klar til at genoptage deres arbejde øjeblikkeligt
English: they are ready to start work again immediately
German: sie sind bereit ihre arbeit unverzüglich wieder aufzunehmen
Finnish: he ovat valmiita ryhtymään työhön välittömästi
Spanish: están listos para empezar a trabajar de inmediato
Mystery: pgk q hgr zhe op gidc wpzh vdwlr izjk f t acy mtc
------------------------------------------------------------
```

3.1.5 Collation and padding for variable-length sequences

In our translation dataset, each source and target sentence can have a different length. However, PyTorch models typically expect a batch to be a single tensor of shape `(batch_size, max_seq_len, ...)`, where all sequences in the batch are padded to the same length.

PyTorch's `DataLoader` handles batching by:

1. Calling `__getitem__` on the dataset multiple times to get a list of individual samples.
2. Passing that list to a collate function, which decides how to turn the list into a single batch.

By default, the collate function just stacks tensors, which doesn't work for variable-length sequences. Instead, we define a custom `pad_collate` that:

- Takes a list of `(source_tensor, target_tensor)` pairs.
- Pads all source sequences in the batch to the same length.
- Pads all target sequences in the batch to the same length.
- Returns two tensors: `padded_source` and `padded_target`, using `PADDING_IDX` as the padding value.

We'll pass this function as `collate_fn` to our `DataLoader` so that every batch is properly padded for our seq2seq model.

[ ]:
```python
# Since this is for machine translation, we pad both source and target sequences
def pad_collate(
    batch: List[Tuple[torch.Tensor, torch.Tensor]]
) -> Tuple[torch.Tensor, torch.Tensor]:
    """
    Collate function for variable-length (source, target) pairs.

    Args:
        batch: A list of samples from the dataset, where each sample is a tuple
               (source_indices, target_indices) of 1D tensors.

    Returns:
        padded_source: 2D tensor of shape (batch_size, max_src_len) with
                       all source sequences padded to the same length.
        padded_target: 2D tensor of shape (batch_size, max_tgt_len) with
                       all target sequences padded to the same length.

    Both use PADDING_IDX to fill in positions beyond the true sequence length.
    """
    # Unzip list of (source, target) into two lists
    source, target = zip(*batch)

    # Pad source and target sequences independently along the time dimension
    padded_source = pad_sequence(
        source,
        batch_first=True,
        padding_value=PADDING_IDX,
    )
    padded_target = pad_sequence(
        target,
        batch_first=True,
        padding_value=PADDING_IDX,
    )

    return padded_source, padded_target
```

3.1.6 Building DataLoaders for training, validation, and testing

Now that we have:

- `TranslationDataset` objects (`train_ds`, `val_ds`, `test_ds`) that return `(source_indices, target_indices)` pairs, and
- a custom `pad_collate` function that pads variable-length sequences in a batch,

we can wrap each dataset in a PyTorch `DataLoader`.

The `DataLoader` is responsible for:

- Sampling indices from the dataset (optionally in random order for training).
- Calling `__getitem__` to fetch individual samples.
- Passing the list of samples to `collate_fn` (here, `pad_collate`) to turn them into a single batched tensor of source sequences and a batched tensor of target sequences.

Below, we create three dataloaders for the English → Spanish experiment: one for training, one for validation, and one for testing, all using a batch size of 32 and our custom collation function.

See the docs for more details.

[ ]:
```
# DataLoaders for English -> Spanish translation
train_dl = DataLoader(
    train_ds,
    batch_size=32,
    shuffle=True,
    collate_fn=pad_collate,
)

val_dl = DataLoader(
    val_ds,
    batch_size=32,
    shuffle=True,
    collate_fn=pad_collate,
)

test_dl = DataLoader(
    test_ds,
    batch_size=32,
    shuffle=True,
    collate_fn=pad_collate,
)
```

3.1.7 LSTM encoder for the source sentence (with packed sequences)

The encoder is the first component of our seq2seq + attention model. It reads the padded source sequence and produces:

- A sequence of encoder hidden states of shape `(batch_size, T_enc, hidden_size)` (one vector per time step up to the padded length).
- A final hidden state and cell state that summarize the entire source sentence; these will be used to initialize the decoder.

We implement the encoder as an LSTM-based `nn.Module` with an `Embedding` layer followed by an `LSTM` (see the PyTorch LSTM docs). The key extra ingredient is how we handle padding with *packed* sequences.

Why do we "pack" sequences? Our inputs are padded to a common length so they fit into a single tensor, but:

- Different sentences in a batch have different true lengths.
- The trailing `<PAD>` tokens are not real data and should not influence the LSTM's hidden states.

Conceptually, a *packed sequence* is just a padded batch plus the true lengths, wrapped in a special object so PyTorch knows which time steps are real tokens and which are padding. This gives us two benefits at once:

1. **Efficiency**
   When we call `pack_padded_sequence`, PyTorch creates a compact representation that lets the LSTM skip over the padded positions. It doesn't waste computation on the trailing `<PAD>` tokens in each sequence.

2. **Correct handling of variable-length sequences**
   Because we also pass in the true lengths, PyTorch knows exactly where each sequence actually ends. That means:

   - The LSTM's final hidden state for each example corresponds to the last *real* token, not to a padded time step.
   - The hidden states at padded positions are effectively ignored, rather than being treated as if they were part of the sentence.

Concretely, in the encoder we:

1. Start with an embedded, padded batch `embeds` of shape `(batch_size, max_seq_len, hidden_size)`.
2. Compute a length for each sequence (the number of non-padding tokens).
3. Call `pack_padded_sequence(embeds, lengths, batch_first=True, enforce_sorted=False)` to produce a packed representation.
4. Feed this packed representation into the LSTM.

After the LSTM, we use `pad_packed_sequence` to convert the packed output back into a regular padded tensor of encoder hidden states with shape `(batch_size, T_enc, hidden_size)`, so that:

- The attention mechanism can attend over all encoder time steps in a familiar `(batch, time, hidden)` layout.
- The rest of our code can treat encoder outputs as a standard padded batch.

Packing lets us work with variable-length sequences in a batch-friendly way: we keep the convenience of padded tensors, while telling PyTorch "these are the real tokens, these are padding! Please ignore the padding for both computation and sequence boundaries."

[ ]:
```python
class EncoderLSTM(nn.Module):
    """
    LSTM-based encoder for the source sequence.

    This module:
      - Embeds token indices into dense vectors.
      - Runs them through an LSTM to produce a sequence of hidden states.
      - Returns the full sequence of encoder outputs plus the final hidden/cell
        states, which will be used to initialize the decoder and drive␣
↪attention.
```

```
    """

    def __init__(
        self,
        input_size: int,
        hidden_size: int,
        padding_idx: int,
        dropout: float = 0.3,
    ) -> None:
        """
        Args:
            input_size:  Size of the source vocabulary (number of embeddings).
            hidden_size: Dimensionality of both the embeddings and LSTM hidden␣
↪state.
            padding_idx: Index used for the <PAD> token (to be ignored in␣
↪embeddings).
            dropout:     Dropout probability applied to the embeddings.
        """
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.padding_idx = padding_idx

        # Map token indices → dense vectors
        self.embedding = nn.Embedding(
            num_embeddings=input_size,
            embedding_dim=hidden_size,
            padding_idx=padding_idx,
        )

        # LSTM over the embedded sequence, with batch dimension first
        self.lstm = nn.LSTM(
            input_size=hidden_size,
            hidden_size=hidden_size,
            batch_first=True,
        )

        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        input_seqs: torch.Tensor,
    ) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        """
        Encode a batch of padded source sequences.

        Args:
```

```
            input_seqs: LongTensor of shape (batch_size, max_src_len) containing
                        token indices, padded with padding_idx.

        Returns:
            encoder_outputs: Tensor of shape (batch_size, T_enc, hidden_size)
                             with LSTM outputs at each time step (padded).
            hidden:          Tensor of shape (batch_size, hidden_size) with the
                             final hidden state for each sequence.
            cell:            Tensor of shape (batch_size, hidden_size) with the
                             final cell state for each sequence.
        """
        # Compute sequence lengths by counting non-padding tokens
        # (assumes padding_idx == 0, so nonzero positions are real tokens)
        lengths = as_tensor([torch.count_nonzero(x) for x in input_seqs])

        # Embed tokens and apply dropout
        embeds = self.dropout(self.embedding(input_seqs))

        # Pack the embedded sequences so the LSTM can skip padding positions
        packed_embeds = nn.utils.rnn.pack_padded_sequence(
            input=embeds,
            lengths=lengths,
            batch_first=True,
            enforce_sorted=False,  # we don't require the batch to be␣
↪length-sorted
        )

        # Run the packed sequence through the LSTM
        packed_output, (hidden, cell) = self.lstm(packed_embeds)

        # Unpack back to a padded tensor of shape (batch_size, T_enc,␣
↪hidden_size)
        encoder_outputs, _ = nn.utils.rnn.pad_packed_sequence(
            packed_output,
            batch_first=True,
            padding_value=self.padding_idx,
            # Ensure we pad back to the original max length in input_seqs
            total_length=input_seqs.size(1),
        )

        # LSTM returns hidden/cell with shape (num_layers * num_directions,␣
↪batch, hidden_size)
        # Here we use a single layer, unidirectional LSTM, so we can squeeze␣
↪the first dim.
        hidden = hidden.squeeze(0)  # (batch_size, hidden_size)
        cell = cell.squeeze(0)      # (batch_size, hidden_size)
```

```
        # Return all encoder states plus the final hidden/cell states
        return encoder_outputs, hidden, cell
```

3.1.8 Dot-product attention over encoder states

The attention layer lets the decoder "look back" at the encoder's hidden states when generating each target token.

In this notebook we implement a simple dot-product attention mechanism:

- The decoder produces a sequence of *query* vectors of shape `(batch_size, T_dec, H)`.
- The encoder provides a sequence of *value* (and implicitly key) vectors of shape `(batch_size, T_enc, H)`.
- For each decoder time step, we compute a similarity score between the decoder query and every encoder state using a dot product.
- We then turn these scores into a probability distribution over encoder positions using `softmax`, giving us attention weights.
- Finally, we take a weighted sum of the encoder states to form a context vector for each decoder time step, of shape `(batch_size, T_dec, H)`.

This is sometimes called "encoder–decoder" or "cross-attention": the queries come from the decoder, the keys/values come from the encoder.

Why implement our own attention layer? PyTorch does have a built-in `nn.MultiheadAttention` module, which is designed for Transformer-style self-attention and multi-head attention. However, for a classic seq2seq-with-attention model:

- We only need single-head, dot-product attention.
- The shapes and mask semantics are simpler and slightly different from the Transformer API.
- Implementing it ourselves makes the computation more transparent: we can directly see how scores, masks, softmax, and context vectors are computed.

So here we define a small `DotProductAttention` module that:

1. Computes attention scores via batch matrix multiplication (`bmm`).
2. Optionally applies a mask so that padded encoder time steps get zero attention.
3. Uses `softmax` to obtain attention weights.
4. Produces context vectors by taking a weighted sum over encoder states.

[ ]:
```
class DotProductAttention(nn.Module):
    """
    Simple dot-product attention over encoder states.

    Given:
      - query:  [B, T_dec, H]   (decoder representations)
      - values: [B, T_enc, H]   (encoder representations; also serve as keys)
      - mask:   [B, T_enc]      (True for real tokens, False for padding)

    # where
    #   B      = batch size
```

```
    #   T_enc  = length of the encoder sequence (number of source time steps)
    #   T_dec  = length of the decoder sequence (number of target time steps)
    #   H      = hidden size / embedding dimension

    This module computes:
      - attention scores between each decoder time step and all encoder time␣
↪steps
      - a probability distribution over encoder positions (attention weights)
      - a context vector for each decoder time step: weighted sum of encoder␣
↪states

    Returns:
      context: [B, T_dec, H]
    """

    def __init__(self) -> None:
        super().__init__()

    def forward(
        self,
        query: torch.Tensor,
        values: torch.Tensor,
        mask: torch.Tensor | None = None,
    ) -> torch.Tensor:
        """
        Args:
            query: Tensor of shape [B, T_dec, H]
            values: Tensor of shape [B, T_enc, H]
            mask: Optional boolean tensor of shape [B, T_enc],
                  where True indicates a real (non-padding) encoder token and
                  False indicates padding that should receive zero attention.

        Returns:
            context: Tensor of shape [B, T_dec, H] containing one context
                     vector per decoder time step.
        """

        # Compute raw attention scores via dot product:
        #   scores[b, i, j] = <query[b, i, :], values[b, j, :]>
        # query:  [B, T_dec, H]
        # values: [B, T_enc, H]
        # values.transpose(1, 2): [B, H, T_enc]
        # scores: [B, T_dec, T_enc]
        # torch.bmm() is the batch matrix-matrix product
        scores = torch.bmm(query, values.transpose(1, 2))

        if mask is not None:
```

```
            # mask is [B, T_enc] with True for real tokens, False for padding.
            # We need to broadcast it to [B, 1, T_enc] so it can be applied
            # across all decoder time steps (T_dec).
            mask_expanded = mask.unsqueeze(1)  # [B, 1, T_enc]

            # Wherever mask_expanded is False (i.e., padding), set scores to a
            # very large negative number so softmax ~ 0 at those positions.
            scores = scores.masked_fill(~mask_expanded, -1e9)

        # Convert scores into attention weights with softmax over encoder time␣
↪steps
        # attn_weights[b, i, :] is a probability distribution over encoder␣
↪positions
        # for decoder time step i.
        attn_weights = torch.softmax(scores, dim=-1)  # [B, T_dec, T_enc]

        # Compute context vectors as a weighted sum of encoder states:
        #   context[b, i, :] = sum_j attn_weights[b, i, j] * values[b, j, :]
        # attn_weights: [B, T_dec, T_enc]
        # values:       [B, T_enc, H]
        # context:      [B, T_dec, H]
        context = torch.bmm(attn_weights, values)

        return context
```

3.1.9 LSTM decoder for the target sentence

The decoder is the second major component of our seq2seq + attention model. It generates the target sentence one time step at a time, using:

- The final hidden and cell states from the encoder as its initial state.
- The previous target tokens (teacher forcing during training) as inputs.
- The attention layer to look back at the encoder's hidden states and form a context vector at each decoding step.

In this notebook, the decoder is implemented as an LSTM-based `nn.Module` with:

- An `Embedding` layer over the target vocabulary.
- An `LSTM` that takes embedded target tokens and produces a sequence of decoder hidden states of shape `(batch_size, T_dec, hidden_size)`.

These decoder hidden states serve as the queries for our dot-product attention module, which will combine them with the encoder outputs. The combined information (decoder state + context) is then used to predict the next target token.

The decoder's `forward` method:

1. Takes a batch of target sequences `input_seqs` (typically shifted right for teacher forcing).
2. Embeds them with the target embedding layer.

3. Runs the embeddings through the LSTM, initialized with the encoder's final hidden and cell states.
4. Returns:
    - The full sequence of decoder outputs over time (to be used as attention queries),
    - The final hidden state,
    - The final cell state.

[ ]:
```
class DecoderLSTM(nn.Module):
    """
    LSTM-based decoder for the target sequence.

    This module:
      - Embeds target token indices into dense vectors.
      - Runs an LSTM over the embedded target sequence, initialized from the
        encoder's final hidden and cell states.
      - Returns the sequence of decoder hidden states plus the final hidden/
↪cell.

    The per-time-step decoder hidden states will later be used as queries for
    the attention mechanism over the encoder outputs.
    """

    def __init__(
        self,
        input_size: int,
        hidden_size: int,
        padding_idx: int,
        dropout: float = 0.0,
    ) -> None:
        """
        Args:
            input_size:  Size of the target vocabulary (number of embeddings).
            hidden_size: Dimensionality of both embeddings and LSTM hidden␣
↪state.
            padding_idx: Index of the <PAD> token in the target vocabulary.
            dropout:     Dropout probability applied to target embeddings.
        """
        super().__init__()
        self.input_size = input_size
        self.hidden_size = hidden_size
        self.padding_idx = padding_idx

        # Target-side embedding layer (for target token indices)
        self.embedding = nn.Embedding(
            num_embeddings=input_size,
            embedding_dim=hidden_size,
            padding_idx=padding_idx,
```

```
        )

        # LSTM that decodes the target sequence, step by step
        self.lstm = nn.LSTM(
            input_size=hidden_size,
            hidden_size=hidden_size,
            batch_first=True,
        )

        self.dropout = nn.Dropout(dropout)

    def forward(
        self,
        input_seqs: torch.Tensor,
        hidden_init: torch.Tensor,
        cell_init: torch.Tensor,
    ) -> Tuple[torch.Tensor, torch.Tensor, torch.Tensor]:
        """
        Args:
            input_seqs: LongTensor of shape [B, T_dec] containing target token
                        indices for the entire decoded sequence (e.g., with EOS
                        shifted for teacher forcing).
            hidden_init: FloatTensor of shape [B, H] with the initial hidden
                         state (typically the encoder's final hidden state).
            cell_init:   FloatTensor of shape [B, H] with the initial cell state
                         (typically the encoder's final cell state).

        Returns:
            output: Tensor of shape [B, T_dec, H] containing decoder hidden
                    states at each time step (used as attention queries).
            hidden: Tensor of shape [B, H] with the final hidden state.
            cell:   Tensor of shape [B, H] with the final cell state.
        """
        # Embed target token indices and apply dropout
        embeds = self.dropout(self.embedding(input_seqs))
        # embeds: [B, T_dec, H]

        # LSTM expects initial states with shape [num_layers, B, H],
        # so we add a layer dimension via unsqueeze(0).
        output, (hidden, cell) = self.lstm(
            embeds,
            (hidden_init.unsqueeze(0), cell_init.unsqueeze(0)),
        )

        # output: [B, T_dec, H]
        # hidden: [1, B, H] → squeeze to [B, H]
        # cell:   [1, B, H] → squeeze to [B, H]
```

```
        return output, hidden.squeeze(0), cell.squeeze(0)
```

3.1.10 Putting it all together: Seq2Seq with attention

We now combine the encoder, decoder, and attention mechanism into a single `Seq2Seq` model.

At a high level:

- The **encoder** (`EncoderLSTM`) reads the source sentence and returns:
  - A sequence of encoder hidden states of shape `(B, T_enc, H)`.
  - A final hidden and cell state of shape `(B, H)`, which summarize the source.
- The **decoder** (`DecoderLSTM`) takes:
  - The previous target tokens (teacher forcing during training),
  - The initial hidden and cell states from the encoder,
  - And produces a sequence of decoder hidden states of shape `(B, T_dec, H)`.
- The **attention layer** (`DotProductAttention`) uses decoder states as queries and encoder states as values, to compute:
  - A context vector for each decoder time step, also of shape `(B, T_dec, H)`.

We then concatenate the decoder hidden state and the attention context at each time step, project them back down to size `H`, and use a final linear layer to predict a distribution over the target vocabulary.

Training: `forward` with teacher forcing During training, we use *teacher forcing*:

- Inputs:
  - `input_seqs`: padded source sentences of shape `(B, T_enc)`.
  - `output_seqs`: padded target sentences of shape `(B, T_dec + 1)` (including start and end markers).
- We feed `output_seqs[:, :-1]` into the decoder as inputs (everything except the last token), and train the model to predict `output_seqs[:, 1:]` (everything except the first token).
- We:
  1. Encode the source.
  2. Decode the (shifted) target with initial states from the encoder.
  3. Apply attention over encoder outputs for each decoder time step.
  4. Project to vocabulary logits.
  5. Mask out padding positions and return:
     - `preds`: a flattened tensor of shape `(M, V)` with logits for all non-pad target positions.
     - `targs`: a flattened tensor of shape `(M,)` with the corresponding true target token IDs.

Here, `M` is the total number of *non-padding* target tokens in the batch, and `V` is the target vocabulary size.

Inference: `generate` with greedy decoding At inference time, we don't know the true target sequence, so we can't use teacher forcing. Instead, we:

1. Encode a single source sentence (shape `(T_enc,)`).
2. Start the decoder with a designated start token (in this notebook, we reuse `<EOS>` as the first token).

3. At each step:
    - Run one decoder step given the previous token and the current hidden/cell state.
    - Compute attention over the encoder outputs to get a context vector.
    - Predict a distribution over the next token, take the argmax (greedy decoding).
    - Append the predicted token and feed it back in as the next input.
4. Stop when we generate an `eos_idx` token or when `max_steps` is reached.

The `generate` method implements this loop for a single example (no batching) and returns the generated token IDs as a 1D tensor.

[ ]:
```python
# Notation for shapes used below:
#   B      = batch size
#   T_enc  = length of the encoder (source) sequence
#   T_dec  = length of the decoder (target) sequence used as input
#   H      = hidden size / embedding dimension
#   V      = size of the target vocabulary


class Seq2Seq(nn.Module):
    """
    Full seq2seq model with LSTM encoder, LSTM decoder, and dot-product attention.

    - EncoderLSTM encodes the source sequence into:
        encoder_outputs: [B, T_enc, H]
        hidden, cell:    [B, H]
    - DecoderLSTM takes target inputs (teacher forcing) and encoder final states to
      produce decoder hidden states:
        decoder_outputs: [B, T_dec, H]
    - DotProductAttention computes context vectors from encoder_outputs for each
      decoder time step:
        context: [B, T_dec, H]
    - A final linear layer maps the combined (decoder + context) representation
      to vocabulary logits:
        logits: [B, T_dec, V]
    """

    def __init__(
        self,
        input_vocab_size: int,
        output_vocab_size: int,
        hidden_size: int,
        padding_idx: int,
    ) -> None:
        """
        Args:
            input_vocab_size:  Size of the source vocabulary.
```

```
            output_vocab_size: Size of the target vocabulary.
            hidden_size:       Dimensionality of encoder/decoder hidden states.
            padding_idx:       Index used for the <PAD> token.
        """
        super().__init__()
        self.input_vocab_size = input_vocab_size
        self.output_vocab_size = output_vocab_size
        self.hidden_size = hidden_size
        self.padding_idx = padding_idx

        # Encoder for the source sequence
        self.encoder = EncoderLSTM(
            input_size=input_vocab_size,
            hidden_size=hidden_size,
            padding_idx=padding_idx,
        )

        # Decoder for the target sequence
        self.decoder = DecoderLSTM(
            input_size=output_vocab_size,
            hidden_size=hidden_size,
            padding_idx=padding_idx,
        )

        # Attention over encoder outputs
        self.attn = DotProductAttention()

        # Combine decoder hidden state and attention context: [B, T_dec, 2H] → [B, T_dec, H]
        self.attn_combine = nn.Linear(2 * hidden_size, hidden_size)

        # Final projection from hidden size to target vocabulary size: [B, T_dec, H] → [B, T_dec, V]
        self.linear1 = nn.Linear(
            in_features=hidden_size,
            out_features=output_vocab_size,
        )

    def forward(
        self,
        input_seqs: torch.Tensor,
        output_seqs: torch.Tensor,
    ) -> Tuple[torch.Tensor, torch.Tensor]:
        """
        Forward pass used for training with teacher forcing.

        Args:
```

```
            input_seqs:  LongTensor [B, T_enc] with padded source token indices.
            output_seqs: LongTensor [B, T_dec + 1] with padded target token␣
↪indices,
                         including both start and end markers. We will use
                         output_seqs[:, :-1] as decoder inputs and
                         output_seqs[:, 1:] as targets.

        Returns:
            preds: Tensor [M, V] with logits for all non-padding target␣
↪positions,
                   flattened across batch and time.
            targs: LongTensor [M] with the corresponding true target token IDs
                   (no padding).
        """

        # 1) Encode the source sequence
        # encoder_outputs: [B, T_enc, H]
        # hidden, cell:    [B, H]
        encoder_outputs, hidden, cell = self.encoder(input_seqs)

        # 2) Decode with teacher forcing
        # Use all but the last target token as input to the decoder.
        # output_seqs shape: [B, T_dec + 1]
        # decoder_inputs:    [B, T_dec]
        decoder_inputs = output_seqs[:, :-1]

        # decoder_outputs: [B, T_dec, H]
        decoder_outputs, hidden, cell = self.decoder(
            decoder_inputs,
            hidden,
            cell,
        )

        # 3) Attention over encoder outputs for each decoder step
        # enc_mask: [B, T_enc], True for real tokens, False for padding
        enc_mask = input_seqs != self.padding_idx

        # context: [B, T_dec, H]
        context = self.attn(decoder_outputs, encoder_outputs, enc_mask)

        # 4) Combine decoder hidden state and attention context, then project
        # Concatenate along the last dimension: [B, T_dec, H] + [B, T_dec, H] →␣
↪[B, T_dec, 2H]
        combined = torch.cat([decoder_outputs, context], dim=-1)  # [B, T_dec,␣
↪2H]
```

```
        # Nonlinear layer to mix decoder and context information: [B, T_dec,␣
↪2H] → [B, T_dec, H]
        attn_out = torch.tanh(self.attn_combine(combined))  # [B, T_dec, H]

        # Final projection to vocabulary logits: [B, T_dec, H] → [B, T_dec, V]
        out = self.linear1(attn_out)  # [B, T_dec, V]

        # 5) Create a mask over non-padding target positions (for the *inputs*␣
↪we used)
        # We predicted output_seqs[:, 1:], so we mask based on output_seqs[:, :
↪-1].
        # mask: [B, T_dec] with True where the input token is not padding.
        mask = output_seqs[:, :-1] != self.padding_idx

        # Flatten predictions and targets over all non-pad positions:
        # preds: [M, V], targs: [M]
        preds = out[mask]                   # select rows where mask is True
        targs = output_seqs[:, 1:][mask]  # true next token at those positions

        return preds, targs

    @torch.no_grad()
    def generate(
        self,
        source: torch.Tensor,
        max_steps: int,
        eos_idx: int,
    ) -> torch.Tensor:
        """
        Greedy decoding for a single source sentence (no batching).

        Args:
            source:    LongTensor [T_enc] with token indices for a single source
                       sentence (unpadded or padded with padding_idx).
            max_steps: Maximum number of decoding steps before we force a stop.
            eos_idx:   Index of the EOS token in the target vocabulary. We will
                       use this both as the starting token and as the stopping
                       condition.

        Returns:
            generated: LongTensor [T_gen] containing the generated target token
                       indices, including the initial token and (if produced)
                       the final EOS token.
        """
        self.eval()

        # 1) Encode single source sequence (add batch dimension)
```

```
        # source: [T_enc] → [1, T_enc]
        encoder_outputs, hidden, cell = self.encoder(source.unsqueeze(0))
        # encoder_outputs: [1, T_enc, H]
        # hidden, cell:    [1, H]

        # Attention mask over encoder time steps: [1, T_enc]
        enc_mask = (source.unsqueeze(0) != self.padding_idx)

        # We use EOS as the "start" token for the decoder in this notebook
        generated = [eos_idx]

        for _ in range(max_steps):
            # Current input token for the decoder: shape [1, 1]
            inp = torch.tensor(
                [[generated[-1]]],
                dtype=torch.long,
                device=source.device,
            )

            # Run one decoding step: reuse DecoderLSTM's embedding and LSTM
            dec_embeds = self.decoder.embedding(inp)  # [1, 1, H]
            dec_output, (hidden_new, cell_new) = self.decoder.lstm(
                dec_embeds,
                (hidden.unsqueeze(0), cell.unsqueeze(0)),
            )

            # dec_output: [1, 1, H] → squeeze time dimension → [1, H]
            dec_output = dec_output.squeeze(1)

            # 2) Attention for this single time step
            # DotProductAttention expects query [B, T_dec, H], so we add T_dec=1
            context = self.attn(
                dec_output.unsqueeze(1),  # [1, 1, H]
                encoder_outputs,          # [1, T_enc, H]
                enc_mask,                 # [1, T_enc]
            )
            # context: [1, 1, H] → [1, H]
            context = context.squeeze(1)

            # 3) Combine decoder state + context and project to logits
            combined = torch.cat([dec_output, context], dim=-1)  # [1, 2H]
            attn_out = torch.tanh(self.attn_combine(combined))   # [1, H]
            logits = self.linear1(attn_out)                      # [1, V]

            # 4) Pick the most likely next token (greedy decoding)
            predicted_word = torch.argmax(logits, dim=-1)        # [1]
            next_id = predicted_word.item()
```

```
            generated.append(next_id)

            # 5) Update hidden/cell for the next step
            hidden = hidden_new.squeeze(0)  # [1, H] → [H]
            cell = cell_new.squeeze(0)      # [1, H] → [H]

            # Stop if we generated EOS
            if next_id == eos_idx:
                break

        return torch.as_tensor(generated, device=source.device)
```

A quick example of teacher forcing Suppose the true target sentence is:

> the cat ate the cat food

but our model currently thinks dinner should be more exciting and predicts:

> the cat ate the spaghetti

If we **don't** use teacher forcing during training, the model's own predictions get fed back into the decoder at each step. So after predicting `spaghetti`, the next input to the decoder is literally the token `spaghetti`. From there, the model might continue with:

1. Next token: `with`
2. Next token: `meatballs`

so the full prediction drifts to:

> the cat ate the spaghetti with meatballs

Now the model is learning how to finish sentences that *start* with its own mistake, instead of learning how to continue the *correct* sentence.

With teacher forcing, we do something different during training:

- At each time step, we feed the ground-truth previous word as input to the decoder, not the model's last prediction.
- So even if the model predicts `spaghetti` when it should have said `cat`, the *next* input to the decoder is still the true word `cat`, not `spaghetti`.

That means the model is always being trained to complete:

> the cat ate the **cat** …

rather than:

> the cat ate the **spaghetti** …

Teacher forcing helps the model learn good local next-word predictions conditioned on the *correct* history, instead of getting dragged further and further away by its own early mistakes.

3.1.11 Training the Seq2Seq model (with early stopping)

We're now ready to train the seq2seq + attention model on the English → Spanish data.

We'll use:

- The training split to update model parameters.
- The validation split to track performance during training and to select the best model (early stopping).
- The test split later to evaluate translation quality using BLEU.

The helper function `compute_test_loss` runs the model over an entire dataloader (validation or test), collects all predictions and targets, and returns the average loss. This keeps the evaluation logic separate from the main training loop.

The `train` function:

1. Instantiates a `Seq2Seq` model with a hidden size of 256.
2. Uses `CrossEntropyLoss` (ignoring padding indices) as the training objective.
3. Optimizes with Adam.
4. For each epoch:
   - Iterates over `train_dl`, runs the model with teacher forcing, computes the loss, and takes an optimizer step.
   - Computes validation loss on `val_dl` using `compute_test_loss`.
   - Logs both train and validation loss.
   - Tracks the best validation loss seen so far and saves the corresponding model state.
   - Uses a patience-based early stopping criterion: if validation loss hasn't improved for `patience` consecutive epochs, training stops.

At the end, the function restores the best validation model weights and returns:

- The trained `Seq2Seq` model.
- A `losses` dictionary with lists of train and validation loss per epoch, which we can later plot.

[ ]:
```python
@torch.no_grad()
def compute_test_loss(
    model: nn.Module,
    val_dl: DataLoader,
    loss_fn: nn.Module,
) -> float:
    """
    Compute the average loss of a model over a given dataloader
    (typically validation or test).

    Args:
        model:   The seq2seq model to evaluate.
        val_dl:  A DataLoader yielding (input_seqs, output_seqs) batches.
        loss_fn: Loss function, e.g. CrossEntropyLoss(ignore_index=padding_idx).

    Returns:
        The scalar average loss over all batches in the dataloader.
```

```
    """
    model.eval()  # important for dropout, etc.
    all_preds, all_targs = [], []

    for input_seqs, output_seqs in val_dl:
        preds, targs = model(input_seqs, output_seqs)
        all_preds.append(preds)
        all_targs.append(targs)

    preds = torch.cat(all_preds, dim=0)
    targs = torch.cat(all_targs, dim=0)

    return loss_fn(preds, targs).item()

def train(
    train_ds,
    train_dl: DataLoader,
    val_ds,
    val_dl: DataLoader,
    n_epochs: int = 20,
    patience: int = 3,
):
    """
    Train the Seq2Seq + attention model with early stopping.

    Args:
        train_ds:  Training TranslationDataset (used for vocab sizes).
        train_dl:  DataLoader over the training split.
        val_ds:    Validation TranslationDataset (unused here but kept for␣
 ↪symmetry).
        val_dl:    DataLoader over the validation split.
        n_epochs:  Maximum number of training epochs.
        patience:  Number of epochs with no val-loss improvement before␣
 ↪stopping.

    Returns:
        model:  The Seq2Seq model with the best validation loss.
        losses: Dictionary with 'train' and 'val' lists of per-epoch losses.
    """
    # Instantiate seq2seq model
    model = Seq2Seq(
        input_vocab_size=train_ds.source_vocab_size,
        output_vocab_size=train_ds.target_vocab_size,
        hidden_size=256,
        padding_idx=PADDING_IDX,
    )
```

```
# Cross-entropy over vocabulary; ignore padding positions
criterion = CrossEntropyLoss(ignore_index=PADDING_IDX)
optimizer = Adam(model.parameters(), lr=0.001, weight_decay=1e-4)

losses = {"train": [], "val": []}

best_val = float("inf")
best_state = copy.deepcopy(model.state_dict())
best_epoch = -1

epochs_no_improve = 0
report_interval = 80  # how often to print batch-level progress

print("Beginning training")

for epoch in range(n_epochs):
    model.train()
    running = 0.0      # loss accumulated over report_interval batches
    epoch_loss = 0.0   # loss accumulated over the whole epoch

    for i, (src, trg) in enumerate(train_dl):
        # Forward pass with teacher forcing
        preds, targs = model(src, trg)
        loss = criterion(preds, targs)

        # Backpropagation and parameter update
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()

        # Track running and epoch loss
        running += loss.item()
        epoch_loss += loss.item()

        # Periodic progress report
        if (i + 1) % report_interval == 0:
            print(
                f"[epoch {epoch+1}, {i+1:3d} batches] "
                f"loss: {running / report_interval:.4f}"
            )
            running = 0.0

    # Evaluate on validation set
    model.eval()
    with torch.no_grad():
        val_loss = compute_test_loss(model, val_dl, criterion)
```

```
        train_loss = epoch_loss / len(train_dl)
        losses["train"].append(train_loss)
        losses["val"].append(val_loss)

        print(f"Epoch {epoch+1}: train={train_loss:.4f}, val={val_loss:.4f}")

        # Early stopping: track best validation loss
        if val_loss < best_val:
            best_val = val_loss
            best_state = copy.deepcopy(model.state_dict())
            best_epoch = epoch + 1  # 1-based indexing for readability
            epochs_no_improve = 0
        else:
            epochs_no_improve += 1
            if epochs_no_improve >= patience:
                print(f"No improvement for {patience} epochs. Stopping early.")
                break

    # Restore the best model parameters before returning
    model.load_state_dict(best_state)
    print(f"Finished training. Best val loss: {best_val:.4f} at epoch␣
↪{best_epoch}")

    return model, losses
```

3.1.12 Training on English → Spanish

We are now ready to train the seq2seq + attention model on the English → Spanish parallel data. The call below runs the training loop, applies early stopping based on validation loss, and returns:

- `model_en_es`: the trained `Seq2Seq` model (restored to the best validation epoch),
- `losses_en_es`: a dictionary with per-epoch train and validation losses that we can plot later.

[ ]:
```
%%time
# Train the English → Spanish seq2seq model
model_en_es, losses_en_es = train(train_ds, train_dl, val_ds, val_dl)
```

```
Beginning training
[epoch 1,  80 batches] loss: 5.7516
[epoch 1, 160 batches] loss: 5.0289
Epoch 1: train=5.3623, val=4.4723
[epoch 2,  80 batches] loss: 4.5891
[epoch 2, 160 batches] loss: 4.3472
Epoch 2: train=4.4577, val=3.9454
[epoch 3,  80 batches] loss: 4.0093
[epoch 3, 160 batches] loss: 3.9490
Epoch 3: train=3.9738, val=3.6685
[epoch 4,  80 batches] loss: 3.6443
```

```
[epoch 4, 160 batches] loss: 3.6123
Epoch 4: train=3.6243, val=3.4722
[epoch 5,  80 batches] loss: 3.3465
[epoch 5, 160 batches] loss: 3.3163
Epoch 5: train=3.3321, val=3.3759
[epoch 6,  80 batches] loss: 3.0465
[epoch 6, 160 batches] loss: 3.1089
Epoch 6: train=3.0839, val=3.2750
[epoch 7,  80 batches] loss: 2.8317
[epoch 7, 160 batches] loss: 2.8877
Epoch 7: train=2.8570, val=3.2292
[epoch 8,  80 batches] loss: 2.6168
[epoch 8, 160 batches] loss: 2.6966
Epoch 8: train=2.6564, val=3.2144
[epoch 9,  80 batches] loss: 2.4079
[epoch 9, 160 batches] loss: 2.5094
Epoch 9: train=2.4634, val=3.1872
[epoch 10,  80 batches] loss: 2.2167
[epoch 10, 160 batches] loss: 2.3311
Epoch 10: train=2.2797, val=3.2137
[epoch 11,  80 batches] loss: 2.0572
[epoch 11, 160 batches] loss: 2.1335
Epoch 11: train=2.0988, val=3.2315
[epoch 12,  80 batches] loss: 1.8815
[epoch 12, 160 batches] loss: 1.9969
Epoch 12: train=1.9398, val=3.2385
No improvement for 3 epochs. Stopping early.
Finished training. Best val loss: 3.1872 at epoch 9
CPU times: user 8min 18s, sys: 3.81 s, total: 8min 22s
Wall time: 1min 28s
```

3.1.13 Visualizing training and validation loss

To get a quick sense of how well training went, it's helpful to plot both the training and validation loss over epochs. A good model should typically show:

- Training loss decreasing over time.
- Validation loss decreasing at first and then flattening or slightly increasing once the model starts to overfit.

In the plot above, we can see:

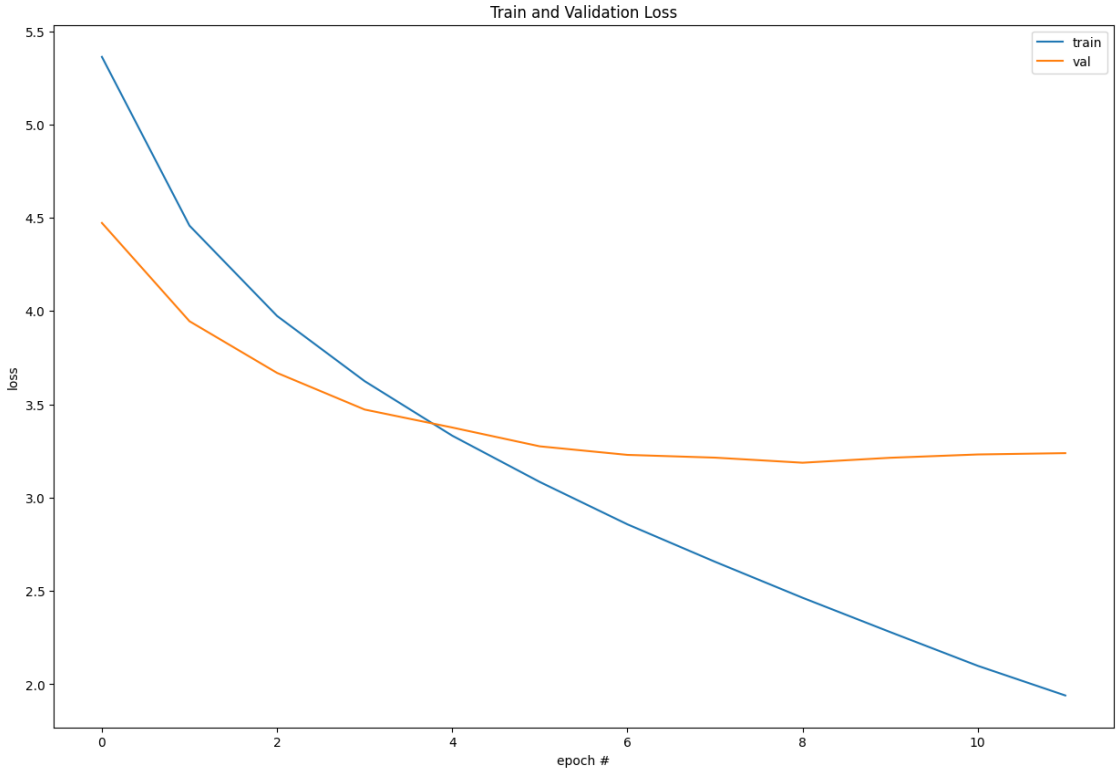
- **Training loss** steadily decreases across all epochs, which tells us the model keeps fitting the training data better and better.
- **Validation loss** drops quickly at the beginning (roughly epochs 1–5), then flattens and starts to wobble around a slightly higher value. The best validation loss occurs around **epoch 10**.

Because we enabled early stopping with `patience=3`, training stops once the validation loss has failed to improve for three consecutive epochs. In this run, that happens after epoch 13, and the code restores the model weights from the best epoch (epoch 10). This is exactly what we want: we

keep the model that generalizes best to unseen data, rather than the one that simply drives the training loss as low as possible.

[ ]:
```
# Plot the training and validation loss curves over epochs
plt.rcParams["figure.figsize"] = [15, 10]

pd.DataFrame(losses_en_es).plot(
    xlabel="epoch #",
    ylabel="loss",
    title="Train and Validation Loss",
)
```

[ ]:
```
<Axes: title={'center': 'Train and Validation Loss'}, xlabel='epoch #',
ylabel='loss'>
```



3.1.14 Evaluating translation quality with BLEU

Loss is useful for monitoring training, but it's not a very interpretable metric for translation quality. To compare different models and checkpoints in a way that roughly matches human judgment, we use **BLEU** (Bilingual Evaluation Understudy), here via the `sacrebleu` library.

At a high level, (unigram) BLEU:

- Compares the word counts in the system output (candidate translation) to the word counts

in one or more human reference translations.
- Uses clipped counts so that a word repeated more times than in the references doesn't get extra credit.
- Aggregates these matches over the whole corpus and applies a **brevity penalty** so that overly short translations are penalized.
- Produces a score between 0 and 1 (or 0 and 100 in some libraries), where higher is better.

Candidate and reference formats The helper `translate_corpus` below translates all sentences in the test set and returns:

- `candidate_text`: the model's translations
  Type: `List[List[str]]`
  Each inner list is a single translated sentence as a list of word tokens, e.g:

  ```
  candidate_text = [
      ["this", "is", "sentence", "one"],
      ["here", "is", "another", "mighty", "fine", "translation"],
  ]
  ```

- `reference_text`: the reference translations from the dataset Type: `List[List[List[str]]]`

  BLEU allows multiple reference translations for each candidate, so there's an extra level of nesting: for each candidate sentence, we have a list of possible references. For example:

  ```
  reference_text = [
      [
          # all reference translations for the first candidate
          ["this", "is", "sentence", "one"],
          ["here", "is", "another", "translation", "for", "sentence", "one"],
      ],
      [
          # all reference translations for the second candidate
          ["here", "is", "another", "mighty", "fine", "translation"],
      ],
  ]
  ```

In our setup, we only have one reference per sentence, so for each candidate you'll see a list containing a single reference:

```
reference_text[i]  # → [ [w1, w2, ..., wK] ]
```

The `print_bleu` function will:

1. Join each candidate token list into a string.
2. Rearrange the references into the format expected by `sacrebleu.corpus_bleu`.
3. Compute the corpus-level BLEU score and print it (normalized to the 0–1 range).

Below this markdown, `translate_corpus` and `print_bleu` implement this whole pipeline using `sacrebleu` instead of `torchtext.bleu_score`.

[ ]:
```
def translate_corpus(
    vocab: Dict[str, int],
```

```
    test_dl: DataLoader,
    model: nn.Module,
) -> Tuple[List[List[str]], List[List[List[str]]]]:
    """
    Run the model on the entire test set and collect candidate and reference
    translations in the format expected by BLEU.

    Args:
        vocab:   Target vocabulary mapping token → index.
        test_dl: DataLoader over the test split, yielding (source_batch,␣
↪target_batch).
        model:   Trained Seq2Seq model with a .generate() method.

    Returns:
        candidate_text: List of candidate translations.
            Shape: [num_sentences][tokens]
        reference_text: List of reference translations.
            Shape: [num_sentences][num_refs][tokens]
            (here num_refs = 1, but BLEU allows multiple references per␣
↪sentence)
    """
    # Invert vocab: index → token
    index_to_word = {v: k for k, v in vocab.items()}

    candidate_text: List[List[str]] = []
    reference_text: List[List[List[str]]] = []

    for source_batch, target_batch in test_dl:
        # Iterate over individual sentence pairs in the batch
        for source, target in zip(source_batch, target_batch):
            # Generate a translation. We cap length at 20 tokens
            # (the max length in this dataset is 14).
            translated_indices = model.generate(
                source,
                max_steps=20,
                eos_idx=EOS_IDX,
            )

            # Map token indices back to word strings
            new_candidate_text = [
                index_to_word[idx.item()] for idx in translated_indices
            ]

            # For references, BLEU expects a list of possible references per
            # sentence, even if there is only one.
            new_reference_text = [
                index_to_word[idx.item()] for idx in target
```

```
            ]
            new_reference_text = [new_reference_text]  # wrap in outer list

            candidate_text.append(new_candidate_text)
            reference_text.append(new_reference_text)

    return candidate_text, reference_text


def print_bleu(
    lang_name: str,
    train_ds,
    test_dl: DataLoader,
    model: nn.Module,
) -> None:
    """
    Compute and print the corpus-level BLEU score using sacrebleu.

    Args:
        lang_name: Human-readable label for the language pair (for printing).
        train_ds:  Training TranslationDataset (used here only for its␣
 ↪target_vocab).
        test_dl:   DataLoader over the test split.
        model:     Trained Seq2Seq model to evaluate.
    """
    cand, refs = translate_corpus(train_ds.target_vocab, test_dl, model)
    # cand: [num_sentences][tokens]
    # refs: [num_sentences][num_refs][tokens]

    # 1) Candidates: join tokens into strings, one per sentence
    sys_stream = [" ".join(s) for s in cand]  # [N]

    # 2) References: arrange into [num_refs][N] for sacrebleu
    num_refs = len(refs[0])  # how many references per sentence (here, 1)
    ref_streams: List[List[str]] = []

    for j in range(num_refs):
        # For reference j, collect the j-th reference for every sentence i
        ref_streams.append(
            [" ".join(refs_i[j]) for refs_i in refs]
        )
    # ref_streams: [R][N], where R = num_refs

    # Compute BLEU with sacrebleu (returns score in [0, 100])
    bleu = sacrebleu.corpus_bleu(sys_stream, ref_streams)
    print(lang_name, bleu.score / 100.0)
```

[ ]:
```
print_bleu("English -> Spanish", train_ds, test_dl, model_en_es)
# Example output:
# English -> Spanish 0.12619606701798783
```

```
English -> Spanish 0.12619606701798783
```

3.1.15 Inspecting example translations (filtered and unfiltered)

BLEU gives us a single summary number, but it's also important to look at *actual translations* to understand what the model is doing well (or badly).

The helper function `show_mixed_examples` prints a small set of English → Spanish examples from the test set in two modes:

1. **Filtered examples**
   - We first filter out cases where the source or reference contain too many `<UNK>` tokens.
   - This gives us "cleaner" examples where the vocabulary is mostly known.
   - For each selected sentence, we print:
     - `SRC (EN)`: the English source sentence
     - `PRED(ES)`: the model's Spanish prediction (generated with `model.generate`)
     - `REF (ES)`: the reference Spanish translation
2. **Unfiltered examples**
   - We then sample some sentences with **no restriction** on `<UNK>` count.
   - This shows how the model behaves on more difficult or rare-word cases.

The helper `toks_from_ids` converts index sequences back into tokens, and removes padding and `<EOS>` markers so we see only "visible" words. Parameters like `max_steps`, `max_unk_filtered`, `n_filtered`, and `n_unfiltered` control how long the generated translations can be and how many examples of each type to display.

[ ]:
```
@torch.no_grad()
def show_mixed_examples(
    train_ds,
    test_dl: DataLoader,
    model: nn.Module,
    max_steps: int = 20,
    max_unk_filtered: int = 1,
    n_filtered: int = 5,
    n_unfiltered: int = 5,
) -> None:
    """
    Print a mix of "clean" and "raw" English → Spanish translations
    from the test set.

    Args:
        train_ds:          Training TranslationDataset (used for vocab lookups).
        test_dl:           DataLoader over the test split.
        model:             Trained Seq2Seq model with .generate().
        max_steps:         Maximum number of decoding steps in generation.
```

```
    max_unk_filtered:  Maximum total number of <UNK> tokens allowed across
                       source and reference for an example to appear in the
                       filtered section.
    n_filtered:        Number of filtered examples to show.
    n_unfiltered:      Number of unfiltered examples to show.

The function prints:
  - A block of "filtered" examples with few <UNK> tokens.
  - A block of "unfiltered" examples with no restrictions.
"""
model.eval()

# Build index → token maps for source (English) and target (Spanish)
src_idx2word = {v: k for k, v in train_ds.source_vocab.items()}
tgt_idx2word = {v: k for k, v in train_ds.target_vocab.items()}

def toks_from_ids(ids: torch.Tensor, idx2word: Dict[int, str]) -> List[str]:
    """
    Convert a 1D tensor of token indices into a list of strings,
    filtering out padding and EOS tokens.
    """
    return [
        idx2word[i.item()]
        for i in ids
        if i.item() not in {PADDING_IDX, EOS_IDX}
    ]

# -------- Filtered examples --------
print(f"=== Filtered examples (max UNK = {max_unk_filtered}) ===")
shown_filtered = 0

for src_batch, tgt_batch in test_dl:
    for src, tgt in zip(src_batch, tgt_batch):
        src_tokens = toks_from_ids(src, src_idx2word)
        ref_tokens = toks_from_ids(tgt, tgt_idx2word)

        # Count unknowns in source + reference
        unk_count = src_tokens.count("<UNK>") + ref_tokens.count("<UNK>")
        if unk_count > max_unk_filtered:
            continue

        # Only generate once we know the example passes the UNK filter
        pred_ids = model.generate(
            src,
            max_steps=max_steps,
            eos_idx=EOS_IDX,
        )
```

```
            pred_tokens = toks_from_ids(pred_ids, tgt_idx2word)

            print("SRC (EN):", " ".join(src_tokens))
            print("PRED(ES):", " ".join(pred_tokens))
            print("REF (ES):", " ".join(ref_tokens))
            print("-" * 60)

            shown_filtered += 1
            if shown_filtered >= n_filtered:
                break

        if shown_filtered >= n_filtered:
            break

    # -------- Unfiltered examples --------
    print("=== Unfiltered examples (no UNK limit) ===")
    shown_unfiltered = 0

    for src_batch, tgt_batch in test_dl:
        for src, tgt in zip(src_batch, tgt_batch):
            src_tokens = toks_from_ids(src, src_idx2word)
            ref_tokens = toks_from_ids(tgt, tgt_idx2word)

            pred_ids = model.generate(
                src,
                max_steps=max_steps,
                eos_idx=EOS_IDX,
            )
            pred_tokens = toks_from_ids(pred_ids, tgt_idx2word)

            print("SRC (EN):", " ".join(src_tokens))
            print("PRED(ES):", " ".join(pred_tokens))
            print("REF (ES):", " ".join(ref_tokens))
            print("-" * 60)

            shown_unfiltered += 1
            if shown_unfiltered >= n_unfiltered:
                break

        if shown_unfiltered >= n_unfiltered:
            break


# Example usage on the English → Spanish model
show_mixed_examples(
    train_ds,
    test_dl,
```

```
    model_en_es,
    max_steps=20,
    max_unk_filtered=1,
    n_filtered=5,
    n_unfiltered=5,
)
```

```
=== Filtered examples (max UNK = 1) ===
SRC (EN): i am confident that parliament will also support him in this
PRED(ES): confío en que el parlamento también también en esta cuestión
REF (ES): confío en que el parlamento también le apoye a este respecto
------------------------------------------------------------
SRC (EN): we are in practice opposed to this report
PRED(ES): estamos en contra de esta propuesta de esta propuesta
REF (ES): de hecho nos oponemos a este informe
------------------------------------------------------------
SRC (EN): we are actually getting <UNK> this year
PRED(ES): estamos <UNK> este asunto <UNK>
REF (ES): este año estamos avanzando realmente
------------------------------------------------------------
SRC (EN): we are far from <UNK> that point
PRED(ES): estamos <UNK> que <UNK> esa cuestión
REF (ES): estamos lejos de ello
------------------------------------------------------------
SRC (EN): we are simply doing our job
PRED(ES): estamos <UNK> nuestra cooperación
REF (ES): estamos simplemente <UNK> nuestra función
------------------------------------------------------------
=== Unfiltered examples (no UNK limit) ===
SRC (EN): i am therefore voting in favour of this extremely important report
PRED(ES): por lo tanto voto a favor de esta resolución en europa
REF (ES): por eso voto a favor de este importante informe
------------------------------------------------------------
SRC (EN): they are ready to start work again immediately
PRED(ES): están dispuestos a <UNK> a nuestros países
REF (ES): están <UNK> para <UNK> a trabajar de inmediato
------------------------------------------------------------
SRC (EN): we are therefore appealing for your help
PRED(ES): por tanto estamos pidiendo una <UNK>
REF (ES): por eso les pedimos ayuda
------------------------------------------------------------
SRC (EN): we are talking about eu citizens
PRED(ES): estamos hablando de una cuestión de seguridad
REF (ES): hablamos de ciudadanos de la ue
------------------------------------------------------------
SRC (EN): they are the big <UNK>
PRED(ES): son los ciudadanos de los <UNK>
```

```
REF (ES): ellos son los grandes <UNK>
------------------------------------------------------------
```

3.1.16 Qualitative look at English → Spanish translations

These examples show a pattern that matches the modest BLEU score:

- On the **filtered** examples, the model usually gets the overall sentence structure, tense, and viewpoint right:
  - `i am confident that parliament will also support him in this`
    → `confío en que el parlamento también también en esta cuestión`
    The model nails the opening `confío en que el parlamento`, but drops the verb *apoyar* and fills in a more generic phrase `en esta cuestión`, with a small repetition error (`también también`).
  - `we are in practice opposed to this report`
    → `estamos en contra de esta propuesta de esta propuesta`
    The polarity and stance are correct (`estamos en contra`), but the model swaps *informe* for *propuesta* and repeats the phrase. It captures the sentiment but not the exact content.
- When `<UNK>` appears, it is often hiding content-heavy verbs or nouns:
  - `we are actually getting <UNK> this year`
    → `estamos <UNK> este asunto <UNK>`
    The model keeps the frame `estamos ... este ...`, but the core meaning is almost entirely eaten by unknowns.
  - `we are simply doing our job`
    → `estamos <UNK> nuestra cooperación`
    The subject and progressive form are correct, but the object drifts from "job" to "cooperation," which changes the meaning.
- On the **unfiltered** examples, the same pattern holds: good scaffolding, but frequent lexical drift:
  - `i am therefore voting in favour of this extremely important report`
    → `por lo tanto voto a favor de esta resolución en europa`
    This is a pretty reasonable paraphrase: the discourse marker (`por lo tanto`), the voting frame (`voto a favor de`), and the overall stance are correct, but `resolución en europa` is a hallucinated detail compared to `este importante informe`.
  - `we are talking about eu citizens`
    → `estamos hablando de una cuestión de seguridad`
    Here the model keeps the `estamos hablando de ...` pattern but replaces the topic ("EU citizens") with a generic "security issue," which is grammatically fine but semantically off.

Overall, the model has learned a decent political/Europarl style template: - It handles subject–verb agreement, tense, and high-frequency function words well. - It often produces fluent, plausible Spanish, but with lexical substitutions, repetitions, and generic phrases that blur the original meaning. - `<UNK>` tokens tend to mask precisely the words that would distinguish a sharp, faithful translation from a vague paraphrase.

This qualitative behavior is consistent with a relatively small dataset and a simple LSTM + attention architecture: the model captures the discourse frame and sentiment reliably, but struggles to consistently pin down the exact content words.

3.2 3. Extending to multiple source languages and the mystery language

Now that we've trained and inspected an English → Spanish model, we can reuse the same architecture to probe the mystery language.

Recall that our pickled datasets contain parallel sentences for:

- Danish
- English
- German
- Finnish
- Spanish
- `mystery`

The idea is:

- For each candidate source language (Danish, English, German, Finnish, Spanish), train a seq2seq + attention model to translate **into** the `mystery` language.
- Compare their BLEU scores and training dynamics: if the mystery language is really just an encoded/codebook version of one of these languages, we expect that language → mystery model to perform noticeably better than the others.

To keep the setup for each language pair consistent, we use a small helper function, `mystery_train_wrapper`, which:

1. Builds `TranslationDataset` objects for:
   - train: `<lang> → mystery`
   - validation: `<lang> → mystery`
   - test: `<lang> → mystery`
2. Shares the same source/target vocabularies across train/val/test for that pair.
3. Wraps each dataset in a `DataLoader` that uses our `pad_collate` function.

We'll call this wrapper separately for each source language (Danish, English, German, Finnish, Spanish) and then train a model for each pair.

[ ]:
```
# Helper to create datasets and dataloaders for <lang> → mystery pairs
def mystery_train_wrapper(
    train_dict: Dict[str, List[List[str]]],
    val_dict: Dict[str, List[List[str]]],
    test_dict: Dict[str, List[List[str]]],
    lang: str,
):
    """
    Construct train/val/test TranslationDataset objects and corresponding
    DataLoaders for a given source language → mystery translation task.
```

```
    Args:
        train_dict: Dictionary for the training split, mapping language name
                    to list of tokenized sentences.
        val_dict:   Dictionary for the validation split (same structure).
        test_dict:  Dictionary for the test split (same structure).
        lang:       Name of the source language (e.g., 'danish', 'english').

    Returns:
        train_ds, val_ds, test_ds: TranslationDataset instances for
                                   <lang> → mystery.
        train_dl, val_dl, test_dl: DataLoaders for those datasets, using
                                   pad_collate and batch_size=32.
    """
    # Train dataset: build vocab from <lang> and 'mystery'
    train_ds = TranslationDataset(
        train_dict[lang],
        train_dict["mystery"],
    )

    # Validation and test datasets reuse the same vocabularies as train_ds
    val_ds = TranslationDataset(
        val_dict[lang],
        val_dict["mystery"],
        source_vocab=train_ds.source_vocab,
        target_vocab=train_ds.target_vocab,
    )
    test_ds = TranslationDataset(
        test_dict[lang],
        test_dict["mystery"],
        source_vocab=train_ds.source_vocab,
        target_vocab=train_ds.target_vocab,
    )

    # DataLoaders with our custom collate function for padding
    train_dl = DataLoader(
        train_ds,
        collate_fn=pad_collate,
        num_workers=0,
        shuffle=True,
        batch_size=32,
    )
    val_dl = DataLoader(
        val_ds,
        collate_fn=pad_collate,
        num_workers=0,
        shuffle=True,
        batch_size=32,
```

```
    )
    test_dl = DataLoader(
        test_ds,
        collate_fn=pad_collate,
        num_workers=0,
        shuffle=True,
        batch_size=32,
    )

    return train_ds, val_ds, test_ds, train_dl, val_dl, test_dl
```

3.2.1 Training mystery-language models for each candidate source language

We're now ready to train a separate seq2seq + attention model for each candidate source language → `mystery`:

- Danish → mystery
- English → mystery
- German → mystery
- Finnish → mystery
- Spanish → mystery

For each language, we:

1. Use `mystery_train_wrapper(...)` to build:
   - `<lang> → mystery` train/validation/test `TranslationDataset` objects, and
   - corresponding `DataLoader`s with our padding collate function.
2. Call the same `train(...)` function we used for English → Spanish to:
   - Train the model with early stopping on validation loss.
   - Record the per-epoch train/validation losses for plotting later.

The code pattern is the same for every language: one block that prepares the datasets/dataloaders with `mystery_train_wrapper`, followed by a call to `train(...)` that returns a trained model and its loss history. Below is the Danish example; similar cells are used for English, German, Finnish, and Spanish.

[ ]:
```
%%time
# Train Danish → mystery model
train_ds_danish, val_ds_danish, test_ds_danish, \
train_dl_danish, val_dl_danish, test_dl_danish = mystery_train_wrapper(
    train_dict,
    val_dict,
    test_dict,
    lang="danish",
)
```

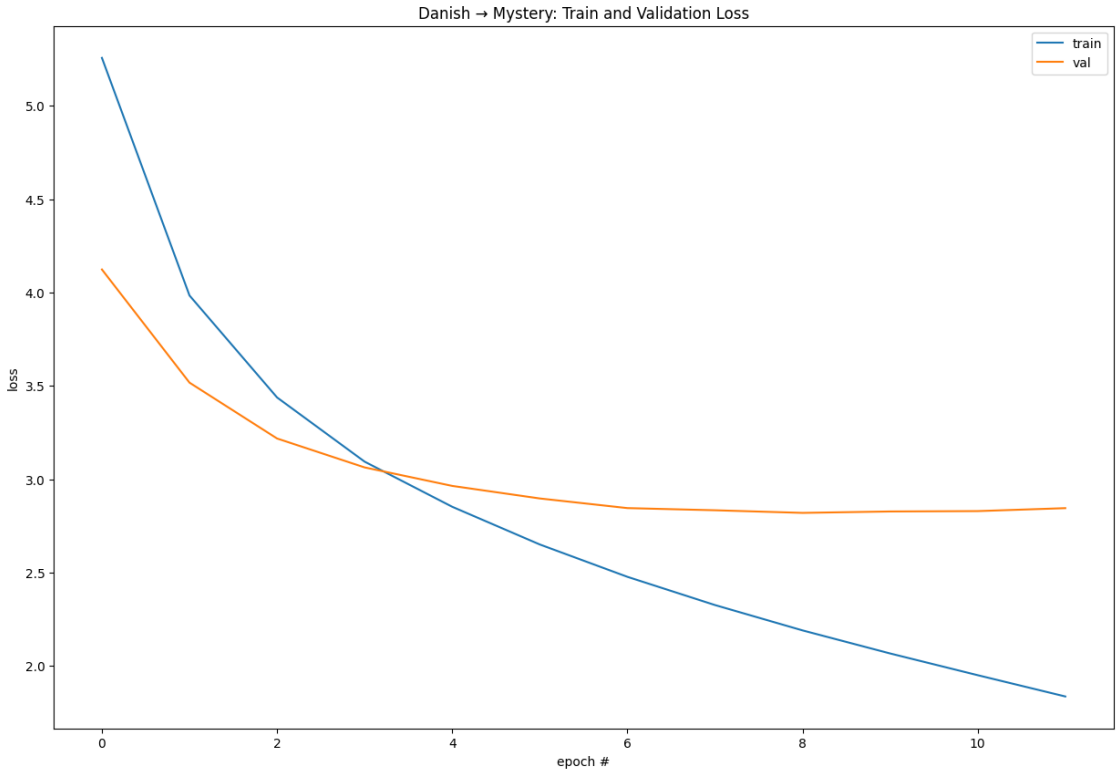
```
model_danish, losses_danish = train(
    train_ds_danish,
    train_dl_danish,
    val_ds_danish,
    val_dl_danish,
)
```

```
Beginning training
[epoch 1,  80 batches] loss: 5.8837
[epoch 1, 160 batches] loss: 4.7115
Epoch 1: train=5.2571, val=4.1238
[epoch 2,  80 batches] loss: 4.1343
[epoch 2, 160 batches] loss: 3.8566
Epoch 2: train=3.9845, val=3.5177
[epoch 3,  80 batches] loss: 3.5117
[epoch 3, 160 batches] loss: 3.3820
Epoch 3: train=3.4383, val=3.2186
[epoch 4,  80 batches] loss: 3.1050
[epoch 4, 160 batches] loss: 3.0838
Epoch 4: train=3.0945, val=3.0637
[epoch 5,  80 batches] loss: 2.8632
[epoch 5, 160 batches] loss: 2.8457
Epoch 5: train=2.8529, val=2.9647
[epoch 6,  80 batches] loss: 2.6188
[epoch 6, 160 batches] loss: 2.6790
Epoch 6: train=2.6510, val=2.8974
[epoch 7,  80 batches] loss: 2.4526
[epoch 7, 160 batches] loss: 2.4947
Epoch 7: train=2.4778, val=2.8459
[epoch 8,  80 batches] loss: 2.3001
[epoch 8, 160 batches] loss: 2.3524
Epoch 8: train=2.3266, val=2.8343
[epoch 9,  80 batches] loss: 2.1512
[epoch 9, 160 batches] loss: 2.2277
Epoch 9: train=2.1908, val=2.8202
[epoch 10,  80 batches] loss: 2.0221
[epoch 10, 160 batches] loss: 2.1067
Epoch 10: train=2.0673, val=2.8278
[epoch 11,  80 batches] loss: 1.8839
[epoch 11, 160 batches] loss: 2.0109
Epoch 11: train=1.9510, val=2.8298
[epoch 12,  80 batches] loss: 1.7713
[epoch 12, 160 batches] loss: 1.8904
Epoch 12: train=1.8370, val=2.8456
No improvement for 3 epochs. Stopping early.
Finished training. Best val loss: 2.8202 at epoch 9
```

```
CPU times: user 10min 56s, sys: 8.27 s, total: 11min 4s
Wall time: 1min 51s
```

[ ]:
```
# Plot Danish → mystery training and validation loss
plt.rcParams["figure.figsize"] = [15, 10]

pd.DataFrame(losses_danish).plot(
    xlabel="epoch #",
    ylabel="loss",
    title="Danish → Mystery: Train and Validation Loss",
)
```

[ ]:
```
<Axes: title={'center': 'Danish → Mystery: Train and Validation Loss'},
xlabel='epoch #', ylabel='loss'>
```



[ ]:
```
%%time
# Train English → mystery model
train_ds_english, val_ds_english, test_ds_english, \
train_dl_english, val_dl_english, test_dl_english = mystery_train_wrapper(
    train_dict,
    val_dict,
    test_dict,
    lang="english",
```

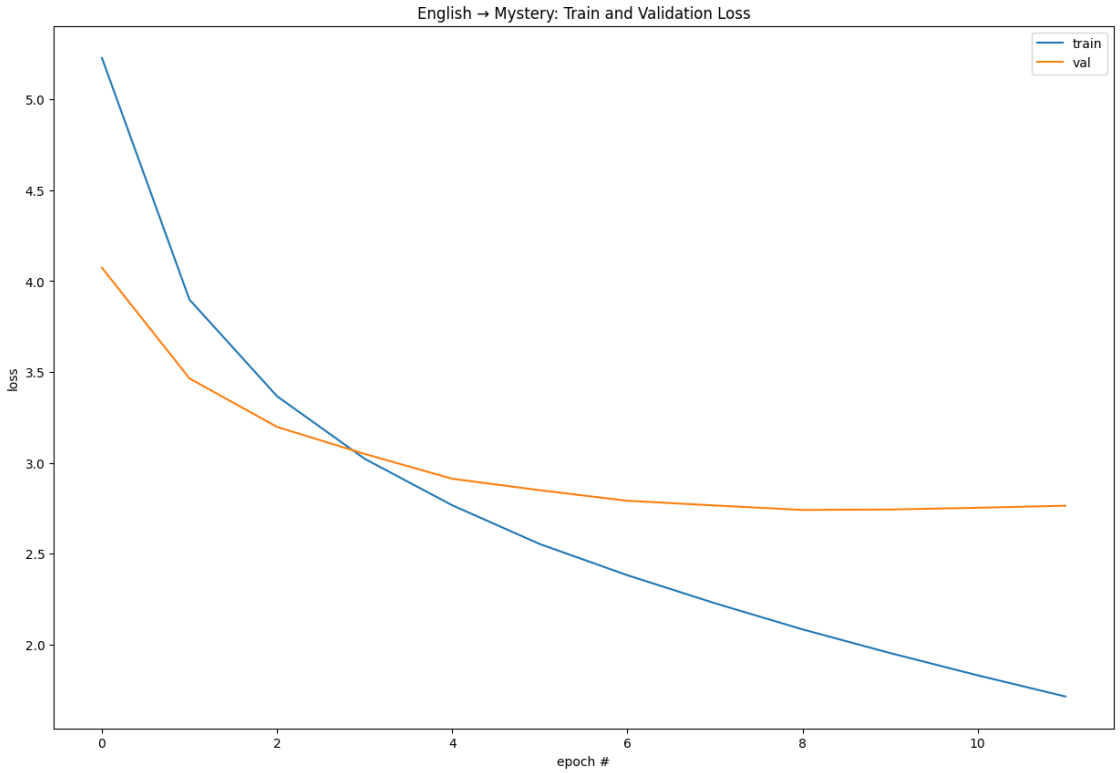
```
)

model_english, losses_english = train(
    train_ds_english,
    train_dl_english,
    val_ds_english,
    val_dl_english,
)
```

```
Beginning training
[epoch 1,  80 batches] loss: 5.8964
[epoch 1, 160 batches] loss: 4.6440
Epoch 1: train=5.2293, val=4.0739
[epoch 2,  80 batches] loss: 4.0398
[epoch 2, 160 batches] loss: 3.7786
Epoch 2: train=3.8982, val=3.4643
[epoch 3,  80 batches] loss: 3.4050
[epoch 3, 160 batches] loss: 3.3394
Epoch 3: train=3.3662, val=3.1974
[epoch 4,  80 batches] loss: 3.0219
[epoch 4, 160 batches] loss: 3.0204
Epoch 4: train=3.0220, val=3.0488
[epoch 5,  80 batches] loss: 2.7428
[epoch 5, 160 batches] loss: 2.7794
Epoch 5: train=2.7660, val=2.9121
[epoch 6,  80 batches] loss: 2.5130
[epoch 6, 160 batches] loss: 2.5941
Epoch 6: train=2.5526, val=2.8488
[epoch 7,  80 batches] loss: 2.3461
[epoch 7, 160 batches] loss: 2.4094
Epoch 7: train=2.3809, val=2.7908
[epoch 8,  80 batches] loss: 2.2042
[epoch 8, 160 batches] loss: 2.2333
Epoch 8: train=2.2266, val=2.7649
[epoch 9,  80 batches] loss: 2.0305
[epoch 9, 160 batches] loss: 2.1275
Epoch 9: train=2.0826, val=2.7403
[epoch 10,  80 batches] loss: 1.8964
[epoch 10, 160 batches] loss: 1.9980
Epoch 10: train=1.9524, val=2.7428
[epoch 11,  80 batches] loss: 1.7827
[epoch 11, 160 batches] loss: 1.8665
Epoch 11: train=1.8297, val=2.7523
[epoch 12,  80 batches] loss: 1.6575
[epoch 12, 160 batches] loss: 1.7572
Epoch 12: train=1.7133, val=2.7635
No improvement for 3 epochs. Stopping early.
```

```
Finished training. Best val loss: 2.7403 at epoch 9
CPU times: user 10min 24s, sys: 9.35 s, total: 10min 33s
Wall time: 1min 45s
```

[ ]:
```
# Plot English → mystery training and validation loss
plt.rcParams["figure.figsize"] = [15, 10]

pd.DataFrame(losses_english).plot(
    xlabel="epoch #",
    ylabel="loss",
    title="English → Mystery: Train and Validation Loss",
)
```

[ ]:
```
<Axes: title={'center': 'English → Mystery: Train and Validation Loss'},
xlabel='epoch #', ylabel='loss'>
```



[ ]:
```
%%time
# Train German → mystery model
train_ds_german, val_ds_german, test_ds_german, \
train_dl_german, val_dl_german, test_dl_german = mystery_train_wrapper(
    train_dict,
    val_dict,
    test_dict,
```

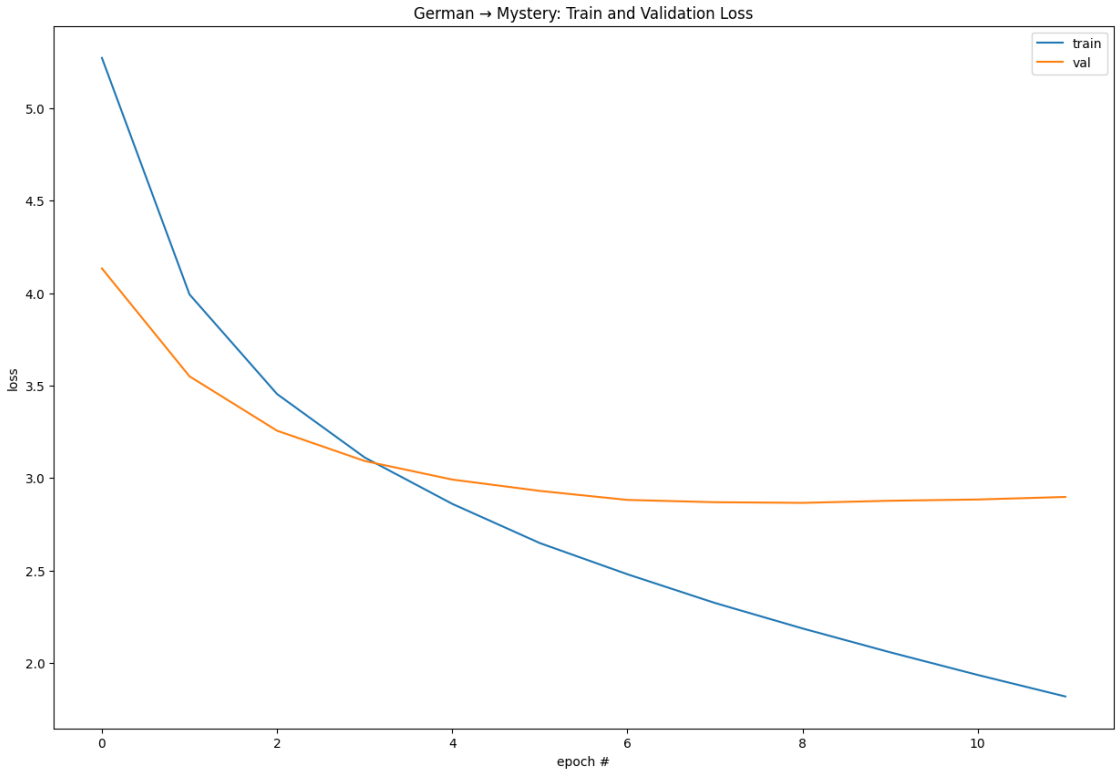
```
    lang="german",
)

model_german, losses_german = train(
    train_ds_german,
    train_dl_german,
    val_ds_german,
    val_dl_german,
)
```

```
Beginning training
[epoch 1,  80 batches] loss: 5.8942
[epoch 1, 160 batches] loss: 4.7213
Epoch 1: train=5.2709, val=4.1332
[epoch 2,  80 batches] loss: 4.1373
[epoch 2, 160 batches] loss: 3.8617
Epoch 2: train=3.9927, val=3.5500
[epoch 3,  80 batches] loss: 3.5219
[epoch 3, 160 batches] loss: 3.4006
Epoch 3: train=3.4542, val=3.2562
[epoch 4,  80 batches] loss: 3.1138
[epoch 4, 160 batches] loss: 3.1154
Epoch 4: train=3.1116, val=3.0926
[epoch 5,  80 batches] loss: 2.8552
[epoch 5, 160 batches] loss: 2.8679
Epoch 5: train=2.8609, val=2.9921
[epoch 6,  80 batches] loss: 2.6413
[epoch 6, 160 batches] loss: 2.6601
Epoch 6: train=2.6489, val=2.9309
[epoch 7,  80 batches] loss: 2.4419
[epoch 7, 160 batches] loss: 2.5151
Epoch 7: train=2.4805, val=2.8821
[epoch 8,  80 batches] loss: 2.2797
[epoch 8, 160 batches] loss: 2.3643
Epoch 8: train=2.3252, val=2.8697
[epoch 9,  80 batches] loss: 2.1377
[epoch 9, 160 batches] loss: 2.2276
Epoch 9: train=2.1875, val=2.8662
[epoch 10,  80 batches] loss: 1.9990
[epoch 10, 160 batches] loss: 2.1090
Epoch 10: train=2.0587, val=2.8776
[epoch 11,  80 batches] loss: 1.8792
[epoch 11, 160 batches] loss: 1.9823
Epoch 11: train=1.9363, val=2.8844
[epoch 12,  80 batches] loss: 1.7692
[epoch 12, 160 batches] loss: 1.8668
Epoch 12: train=1.8200, val=2.8981
```

```
No improvement for 3 epochs. Stopping early.
Finished training. Best val loss: 2.8662 at epoch 9
CPU times: user 11min 10s, sys: 13.8 s, total: 11min 24s
Wall time: 1min 54s
```

[ ]:
```
# Plot German → mystery training and validation loss
plt.rcParams["figure.figsize"] = [15, 10]

pd.DataFrame(losses_german).plot(
    xlabel="epoch #",
    ylabel="loss",
    title="German → Mystery: Train and Validation Loss",
)
```

[ ]:
```
<Axes: title={'center': 'German → Mystery: Train and Validation Loss'},
xlabel='epoch #', ylabel='loss'>
```



[ ]:
```
%%time
# Train Finnish → mystery model
train_ds_finnish, val_ds_finnish, test_ds_finnish, \
train_dl_finnish, val_dl_finnish, test_dl_finnish = mystery_train_wrapper(
    train_dict,
    val_dict,
```

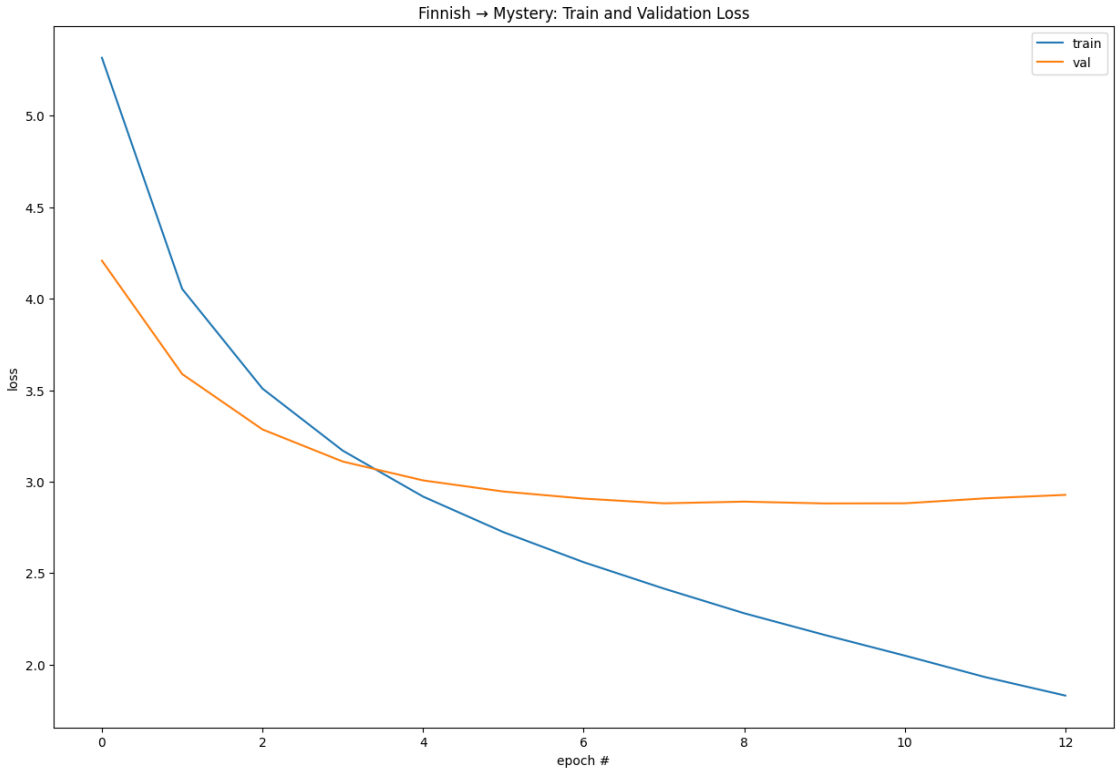
```
    test_dict,
    lang="finnish",
)

model_finnish, losses_finnish = train(
    train_ds_finnish,
    train_dl_finnish,
    val_ds_finnish,
    val_dl_finnish,
)
```

```
Beginning training
[epoch 1,  80 batches] loss: 5.9489
[epoch 1, 160 batches] loss: 4.7667
Epoch 1: train=5.3169, val=4.2081
[epoch 2,  80 batches] loss: 4.1824
[epoch 2, 160 batches] loss: 3.9403
Epoch 2: train=4.0534, val=3.5883
[epoch 3,  80 batches] loss: 3.5856
[epoch 3, 160 batches] loss: 3.4600
Epoch 3: train=3.5089, val=3.2857
[epoch 4,  80 batches] loss: 3.1748
[epoch 4, 160 batches] loss: 3.1600
Epoch 4: train=3.1697, val=3.1101
[epoch 5,  80 batches] loss: 2.9144
[epoch 5, 160 batches] loss: 2.9313
Epoch 5: train=2.9187, val=3.0071
[epoch 6,  80 batches] loss: 2.7000
[epoch 6, 160 batches] loss: 2.7553
Epoch 6: train=2.7245, val=2.9464
[epoch 7,  80 batches] loss: 2.5304
[epoch 7, 160 batches] loss: 2.5953
Epoch 7: train=2.5601, val=2.9077
[epoch 8,  80 batches] loss: 2.3757
[epoch 8, 160 batches] loss: 2.4452
Epoch 8: train=2.4157, val=2.8818
[epoch 9,  80 batches] loss: 2.2429
[epoch 9, 160 batches] loss: 2.3121
Epoch 9: train=2.2809, val=2.8911
[epoch 10,  80 batches] loss: 2.1034
[epoch 10, 160 batches] loss: 2.2136
Epoch 10: train=2.1627, val=2.8811
[epoch 11,  80 batches] loss: 2.0017
[epoch 11, 160 batches] loss: 2.0872
Epoch 11: train=2.0496, val=2.8819
[epoch 12,  80 batches] loss: 1.8915
[epoch 12, 160 batches] loss: 1.9688
```

```
Epoch 12: train=1.9322, val=2.9089
[epoch 13,  80 batches] loss: 1.7778
[epoch 13, 160 batches] loss: 1.8732
Epoch 13: train=1.8310, val=2.9279
No improvement for 3 epochs. Stopping early.
Finished training. Best val loss: 2.8811 at epoch 10
CPU times: user 10min 59s, sys: 8.38 s, total: 11min 8s
Wall time: 1min 51s
```

[ ]:
```python
# Plot Finnish → mystery training and validation loss
plt.rcParams["figure.figsize"] = [15, 10]

pd.DataFrame(losses_finnish).plot(
    xlabel="epoch #",
    ylabel="loss",
    title="Finnish → Mystery: Train and Validation Loss",
)
```

[ ]:
```
<Axes: title={'center': 'Finnish → Mystery: Train and Validation Loss'},
xlabel='epoch #', ylabel='loss'>
```



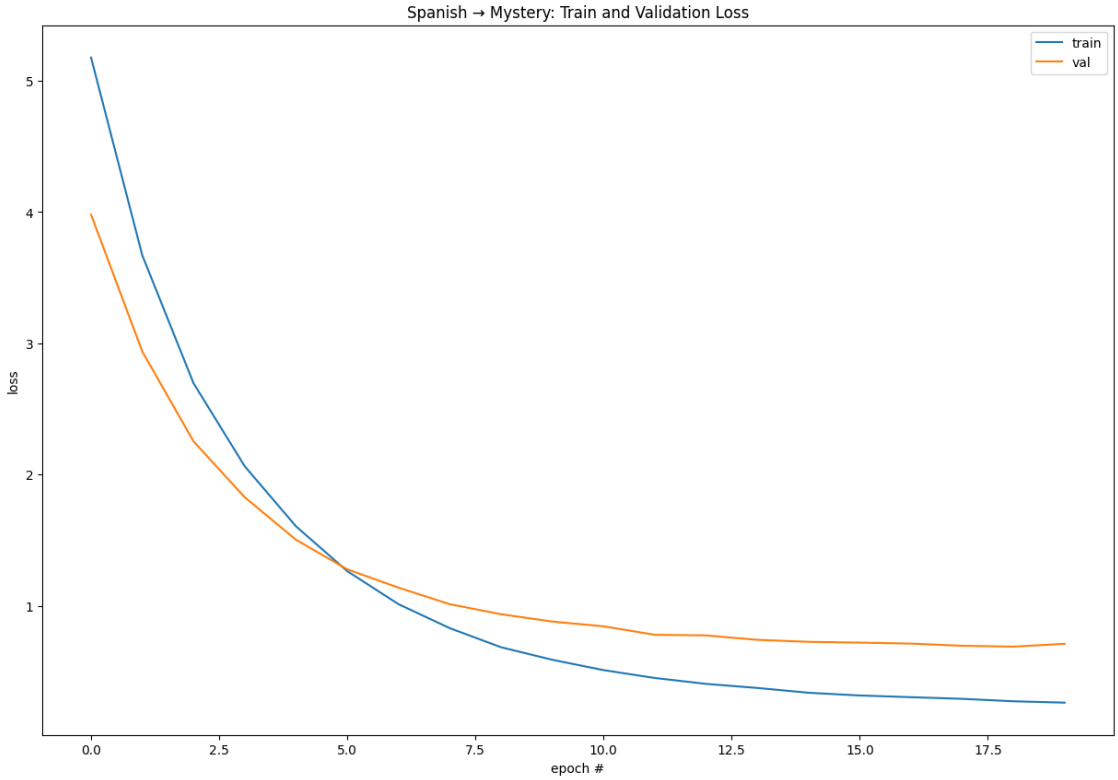
[ ]:
```
%%time
# Train Spanish → mystery model
train_ds_spanish, val_ds_spanish, test_ds_spanish, \
train_dl_spanish, val_dl_spanish, test_dl_spanish = mystery_train_wrapper(
    train_dict,
    val_dict,
    test_dict,
    lang="spanish",
)

model_spanish, losses_spanish = train(
    train_ds_spanish,
    train_dl_spanish,
    val_ds_spanish,
    val_dl_spanish,
)
```

```
Beginning training
[epoch 1,  80 batches] loss: 5.8592
[epoch 1, 160 batches] loss: 4.5975
Epoch 1: train=5.1783, val=3.9811
[epoch 2,  80 batches] loss: 3.9203
[epoch 2, 160 batches] loss: 3.4657
Epoch 2: train=3.6708, val=2.9378
[epoch 3,  80 batches] loss: 2.8515
[epoch 3, 160 batches] loss: 2.5676
Epoch 3: train=2.6959, val=2.2528
[epoch 4,  80 batches] loss: 2.1185
[epoch 4, 160 batches] loss: 2.0202
Epoch 4: train=2.0626, val=1.8268
[epoch 5,  80 batches] loss: 1.6470
[epoch 5, 160 batches] loss: 1.5777
Epoch 5: train=1.6055, val=1.5031
[epoch 6,  80 batches] loss: 1.2861
[epoch 6, 160 batches] loss: 1.2538
Epoch 6: train=1.2640, val=1.2779
[epoch 7,  80 batches] loss: 1.0052
[epoch 7, 160 batches] loss: 1.0175
Epoch 7: train=1.0124, val=1.1386
[epoch 8,  80 batches] loss: 0.8187
[epoch 8, 160 batches] loss: 0.8373
Epoch 8: train=0.8298, val=1.0124
[epoch 9,  80 batches] loss: 0.6579
[epoch 9, 160 batches] loss: 0.7168
Epoch 9: train=0.6843, val=0.9358
[epoch 10,  80 batches] loss: 0.5698
[epoch 10, 160 batches] loss: 0.6048
```

```
Epoch 10: train=0.5888, val=0.8800
[epoch 11,  80 batches] loss: 0.4904
[epoch 11, 160 batches] loss: 0.5266
Epoch 11: train=0.5101, val=0.8438
[epoch 12,  80 batches] loss: 0.4355
[epoch 12, 160 batches] loss: 0.4610
Epoch 12: train=0.4501, val=0.7793
[epoch 13,  80 batches] loss: 0.3822
[epoch 13, 160 batches] loss: 0.4248
Epoch 13: train=0.4052, val=0.7744
[epoch 14,  80 batches] loss: 0.3594
[epoch 14, 160 batches] loss: 0.3860
Epoch 14: train=0.3741, val=0.7407
[epoch 15,  80 batches] loss: 0.3239
[epoch 15, 160 batches] loss: 0.3471
Epoch 15: train=0.3375, val=0.7255
[epoch 16,  80 batches] loss: 0.2977
[epoch 16, 160 batches] loss: 0.3330
Epoch 16: train=0.3167, val=0.7195
[epoch 17,  80 batches] loss: 0.2851
[epoch 17, 160 batches] loss: 0.3160
Epoch 17: train=0.3038, val=0.7122
[epoch 18,  80 batches] loss: 0.2788
[epoch 18, 160 batches] loss: 0.2999
Epoch 18: train=0.2912, val=0.6956
[epoch 19,  80 batches] loss: 0.2637
[epoch 19, 160 batches] loss: 0.2798
Epoch 19: train=0.2726, val=0.6890
[epoch 20,  80 batches] loss: 0.2417
[epoch 20, 160 batches] loss: 0.2823
Epoch 20: train=0.2621, val=0.7105
Finished training. Best val loss: 0.6890 at epoch 19
CPU times: user 18min 19s, sys: 11.5 s, total: 18min 30s
Wall time: 3min 5s
```

[ ]:
```python
# Plot Spanish → mystery training and validation loss
plt.rcParams["figure.figsize"] = [15, 10]

pd.DataFrame(losses_spanish).plot(
    xlabel="epoch #",
    ylabel="loss",
    title="Spanish → Mystery: Train and Validation Loss",
)
```

[ ]:
```
<Axes: title={'center': 'Spanish → Mystery: Train and Validation Loss'},
xlabel='epoch #', ylabel='loss'>
```



3.2.2 Comparing BLEU scores across candidate source languages

Now that we have trained five separate models

- Danish → mystery
- English → mystery
- German → mystery
- Finnish → mystery
- Spanish → mystery

we can evaluate all of them on their respective test sets using corpus-level BLEU.

Intuitively, if the mystery language is really an obfuscated or encoded version of one particular language, the model trained on that true source → mystery pair should achieve a noticeably higher BLEU score than the others. In our case, we expect the Spanish → mystery model to stand out if the mystery text is a codebook version of Spanish.

The cell below computes and prints the BLEU score for each candidate source language, using the same `print_bleu` helper as before.

```
[ ]: # Evaluate BLEU scores for all five source -> mystery models
print_bleu("Danish",  train_ds_danish,  test_dl_danish,  model_danish)
print_bleu("English", train_ds_english, test_dl_english, model_english)
print_bleu("German",  train_ds_german,  test_dl_german,  model_german)
print_bleu("Finnish", train_ds_finnish, test_dl_finnish, model_finnish)
print_bleu("Spanish", train_ds_spanish, test_dl_spanish, model_spanish)
```

```
Danish 0.07872445215517217
English 0.1049584621004643
German 0.07113707172489045
Finnish 0.08096842839696375
Spanish 0.1659926878489357
```

3.2.3 The mystery language is Spanish! With a BLEU score of 0.166, around 53% higher than the next highest language English.

3.2.4 What is the mystery cipher, and why is it lossy?

The `mystery` language in this notebook is not a natural language at all, but a codebook cipher applied to Spanish:

- Each Spanish word type is assigned a random "code" string.
- That code can consist of one or more whitespace-separated tokens (e.g., a single token like `qdy` or a multi-token sequence like `a bism`).
- Sentences are formed by replacing each Spanish word with its corresponding code sequence.

At first glance, this looks like a reversible substitution: if we knew the full codebook, we could just invert the mapping. In practice, several design choices in the cipher make perfect recovery impossible, even with access to the entire corpus.

1. Spaces and segmentation ambiguity Because codes can span multiple tokens and are separated by spaces, the decoder faces a segmentation problem:

- Some codes are single tokens, others are multi-token phrases.
- Worse, a valid code can be the prefix of another valid code, e.g. `a` is a complete code, but `a bism` is also a complete code.

When you scan a mystery sentence from left to right, you must decide whether `a` alone is a code, or whether you should group `a bism` as a longer code. There are hundreds of such prefix overlaps. Any simple "greedy" decoding that always takes the shortest or longest match will make systematic mistakes, and even a smart decoder can't always disambiguate purely from local context.

So the spaces themselves are part of the cipher: they don't cleanly separate words, they separate *code tokens*, and those tokens don't uniquely specify segmentation.

2. Missing entries in the effective codebook Some Spanish words are so rare, or so tightly bound to others, that the data never gives us enough information to distinguish them:

- Certain rare words only ever appear in a fixed collocation (e.g., word A and word B always appear together in the same sentence and never separately).

- On the mystery side, the corresponding code sequence is always seen as a single chunk; we never see A's code or B's code in isolation.

Even with perfect logical "Sudoku-style" elimination, we can get stuck in logical deadlocks: there are multiple assignments of codes to words that are consistent with all observed sentences. In those cases, the codebook cannot be uniquely completed from data alone, so some entries remain effectively undetermined.

3. Collisions: different words, same code The most fundamental problem is that the cipher is not injective:

- In the corpus, we can find explicit collisions, where the *same* mystery code string is aligned to two different Spanish words in different sentences.
- For example (from the analysis notebook), a single code token `q` appears aligned once to *opongo* and once to *responsable*.

This is genuine **information loss**: if both sentences appear as `q` on the mystery side, there is no way—purely from the ciphered text—to know whether the intended word was *opongo* or *responsable* in any new sentence. Any decoder must pick one (usually the more frequent), and will be wrong whenever the rarer word was intended.

Putting it together: a lossy code These three issues interact:

- **Segmentation ambiguity** from multi-token codes and prefix overlaps.
- **Missing or underdetermined entries** for rare / collocated words.
- **True collisions**, where distinct Spanish words share the same code.

Even with a very strong hybrid solver (statistics + logical elimination) and access to all train/val/test data, you can push the recoverable portion of the codebook only to about 96%. The remaining cases require guessing based on context or frequency, not pure inversion of the cipher.

In other words, the mystery cipher is *lossy by design*: it does not preserve enough information for 100% exact reconstruction of the original Spanish text. This explains why, even though Spanish → mystery is clearly the "right" pairing, our seq2seq model's BLEU score can never reach 1.0. The underlying signal itself has been irreversibly compressed and blurred by the codebook.

3.2.5 Comparing training dynamics across language pairs

To get a side-by-side view of how each language pair behaves during training, we can plot the train and validation loss curves for all five source → mystery models in a single grid.

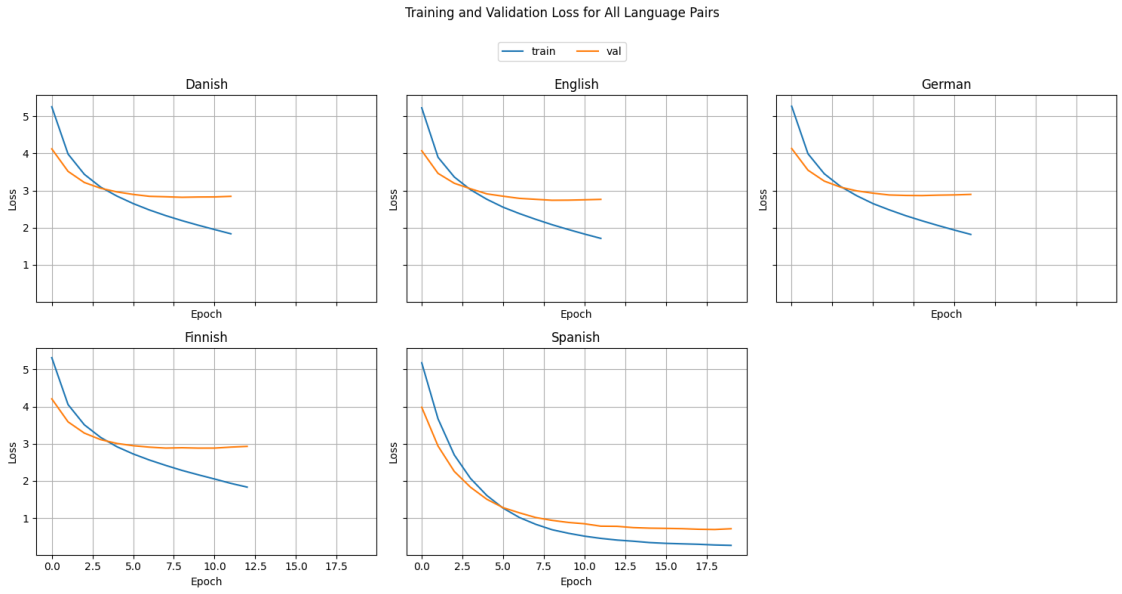
A few patterns are worth noticing:

- For Danish, English, German, and Finnish, the validation loss drops at first and then flattens out around a relatively high value, even as the training loss continues to decrease. This suggests the models are fitting their training data but have limited ability to generalize to the validation set.
- In contrast, the Spanish → mystery model shows both training and validation loss dropping much further and stabilizing at clearly lower values than the other languages.

- The gap between train and validation curves for Spanish is also smaller and more stable, which is what we'd expect when the source language is genuinely well aligned with the mystery language.

Taken together, these training histories already hint that Spanish is the best match for the mystery language, even without looking at BLEU scores.

[ ]:
```
# Compare all 5 language training histories visually in a 2x3 grid
histories = {
    "Danish":  losses_danish,
    "English": losses_english,
    "German":  losses_german,
    "Finnish": losses_finnish,
    "Spanish": losses_spanish,
}

fig, axes = plt.subplots(2, 3, figsize=(15, 8), sharex=True, sharey=True)
axes = axes.ravel()

for ax, (lang, hist) in zip(axes, histories.items()):
    df = pd.DataFrame(hist)  # expects keys "train" and "val"
    ax.plot(df.index, df["train"], label="train")
    ax.plot(df.index, df["val"],   label="val")
    ax.set_title(lang)
    ax.set_xlabel("Epoch")
    ax.set_ylabel("Loss")
    ax.grid(True)

# Hide any unused subplot (since we only have 5 languages)
if len(histories) < len(axes):
    for j in range(len(histories), len(axes)):
        fig.delaxes(axes[j])

# Layout: leave space at top for title + legend
fig.tight_layout(rect=[0, 0, 1, 0.88])

# Shared title
fig.suptitle("Training and Validation Loss for All Language Pairs", y=0.98)

# Shared legend just under the title
handles, labels = axes[0].get_legend_handles_labels()
fig.legend(handles, labels,
           loc="upper center",
           ncol=2,
           bbox_to_anchor=(0.5, 0.93))

plt.show()
```



3.2.6 Why does Spanish → mystery still get the best BLEU?

At this point we know two things:

1. The mystery language is a codebook cipher applied to Spanish.
2. The cipher is lossy: it has segmentation ambiguity, missing or underdetermined entries, and collisions where multiple Spanish words share the same code.

So why does the Spanish → mystery model still achieve a much higher BLEU score than Danish/German/Finnish/English → mystery?

There are a few key reasons.

1. **Forward mapping is consistent, even if inverse mapping is not**

   The codebook is problematic when you try to go from mystery back to Spanish. But in the *forward* direction, Spanish → mystery, the mapping is perfectly well defined:

   - Each Spanish word has a fixed code sequence on the mystery side.
   - Collisions mean that two Spanish words share the same code, but they do not create ambiguity when going forward: both words just map to that same output.

   From the model's perspective, Spanish → mystery is "just" learning a strange but consistent target language. The training data always shows the same Spanish input aligned to the same mystery output, so the model can approximate that mapping quite well.

2. **Other languages must implicitly learn two mappings at once**

   For a non-Spanish source language (say Danish), the underlying generative story is closer to:

   Danish → Spanish (human translation) → cipher(Spanish) = mystery

   But the model only sees:

Danish input → mystery output

To succeed, a Danish → mystery model has to internally approximate both parts of the pipeline:

- A proper translation from Danish into (something like) Spanish.
- The exotic Spanish → code mapping on top of that.

That is strictly harder than learning Spanish → code directly. Any errors in the implicit "Danish → Spanish" step show up as wrong codes on the mystery side, dragging BLEU down.

3. **Word order and structure are already aligned for Spanish**

   The codebook was built on Spanish sentences, so:

   - Word order, morphology, and sentence length of the mystery text are closely tied to Spanish.
   - For Spanish → mystery, the model can often stay near monotonic: source and target positions roughly line up.

   For other languages, the model has to deal with extra reordering and structural differences before it can even try to match the code tokens. That extra complexity again lowers BLEU.

4. **Lossiness caps the ceiling but not the ranking**

   The lossy parts of the cipher (collisions, ambiguous segmentations, rare words) hurt *every* model equally in the sense that even a perfect Spanish → mystery system cannot reach BLEU = 1.0:

   - Whenever two Spanish words share the same code, no model can produce outputs that uniquely preserve the original choice.
   - Ambiguous multi-token codes and underdetermined entries create irreducible uncertainty in the target sequences.

   This effectively lowers the maximum achievable BLEU for the "correct" language pair, but it does not give any advantage to the *wrong* language pairs. Spanish → mystery is still the one that can best match the observed code sequences; the others are fighting both the normal translation problem and the weirdness of the cipher.

The cipher being lossy explains why Spanish → mystery cannot achieve very high absolute BLEU, but the mapping is still far easier to learn from Spanish than from Danish, English, German, or Finnish. That is why Spanish → mystery stands out clearly in both the loss curves and the BLEU scores, even though perfect reconstruction of the original Spanish is provably impossible.